

AI-Powered Chat-Based Academic Advisor for CS Department



Department of Computer Science

Final Year Project (Project 1 – Documentation)

Submitted by: Usman Mustafa

Registration No.: 04072213047

Supervisor: Dr. Mudassar Azam Sindhu

Session (2022-26)

Acknowledgment

To begin with, I want to thank **Almighty Allah** whose blessings, wisdom, and guidance I have always used to be my source of strength in the process of completing this project. This would not have been able to be achieved without His grace. I would also like to thank my respected supervisor **Dr. Mudassar Azam Sindhu** who has provided invaluable supervision, constant motivation, and brilliant suggestions during the progression of this project. His experience, patience and work ethics have inspired and motivated me a lot so as to complete this work successfully.

I would also like to acknowledge an owed debt to the **Department of Computer Science, Quaid-i-Azam University, Islamabad**, to make this project possible and offer me the academic resources, the technical facilities, and a favorable learning environment. I am also thankful to teachers and people who guided me throughout my studies as they assisted me in acquiring the necessary skills and knowledge to pursue this project.

Last but not least, I would like to thank my family and friends, who have been my constant encouragement, moral support and prayers all along the way. They understood and were patient enough to make me believe that I was capable of conquering all the challenges that I was going through.

Abstract

The main task of the project **AI-Powered Chat-Based Academic Advisor to the Computer Science Department at Quaid-i-Azam University (QAU)** is to design a virtual academic advisor capable of properly addressing student inquiries 24 hours a day, without being restricted by office hours and human assistance.

The system affect the **AI and NLP** strength to comprehend and answer student inquiries in dialogic and human-like way. It employs a streamlined and **multi-lingual BERT (Bidirectional Encoder Representations of Transformers) model**, which can interpret and process queries in both English and Urdu, and maintain entire inclusiveness and convenience to every department student.

The chatbot will be able to provide responses to a broad variety of academic questions, such as, but not limited to:

- **Registration of courses** and conditions of eligibility.
- **Distribution of credit hours** per semester.
- **Prerequisites** and sequences of academic progression.
- **Terms of payment** and fees.
- **Examination schedules** and classes.
- **Thesis and research** project guidelines.

The assistant also needs to use structured knowledge base to retrieve information that will be accurate and reliable to the responses and that will be built with official departmental documents such as regulations, syllabi, academic calendars and policy manuals. The system is designed to be dynamically updated and it can easily integrate new policies or changes in the curriculum.

Table of Contents

Acknowledgment	2
Abstract.....	3
List of Figures.....	7
List of Tables	8
CHAPTER - 01	9
SOFTWARE PROJECT MANAGEMENT PLAN.....	9
1.1-Introduction	9
1.2-Problem Definition.....	9
1.3 Proposed System.....	9
1.4 Scope	11
1.5 Objectives	12
1.6 Motivation.....	12
1.7 Risk & Contingencies	12
Project Schedule	13
1.8 Application	14
1.9 Comparative Analysis of Existing Local Systems	14
1.10 Project Organization	14
Process Model	14
CHAPTER – 02	16
SOFTWARE REQUIREMENTS SPECIFICATION.....	16
2.1 Introduction	16
2.2 Product Overview	16
2.3 Specific Requirements	16
2.4 Software Product Features	17
2.5 Software System Attributes.....	17
2.6 List of Use Cases	18
2.7 Use Case Diagram	18
2.7.1 Use Case Descriptions	21
2.7.2 System Sequence Diagram	26
2.7.3 Sequence Diagram.....	36

CHAPTER – 03	46
SOFTWARE DESIGN DESCRIPTION.....	46
3.1 Introduction	46
Design Overview	46
How Components Interact	46
Requirements Traceability Matrix	46
Requirements Traceability Matrix (RTM) for AI-Powered Chat-Based Academic Advisor	47
3.2 System Architecture Design	48
3.2.1 Chosen System Architecture	48
1. Presentation Layer	48
2. Application Layer.....	49
3. NLP Processing Layer	49
4. Data Layer	50
3.3 Architectural Diagram.....	50
Figure 3.1: Architecture Diagram of Academic Advisor	50
3.4 Class Diagram.....	51
3.5 Component Description.....	53
3.6 Activity Diagrams	54
3.7 Interface/Screen Design	55
3.7.1 Login Screen.....	55
3.7.2 Ask Academic Query	55
3.7.3 View Query History	56
3.7.4 Admin Login and Dashboard	56
3.7.5 Admin Manage Data	57
3.7.6 Admin Monitor Activity	58
3.8 Summary	58
CHAPTER: 04.....	59
SOFTWARE TEST DOCUMENTATION	59
4.1 Introduction	59
4.1.1 System Overview	59
4.1.2 Test Approach.....	59
4.1.3 Features to be Tested	60
4.1.4 Features Not to be Tested	60
4.2 Testing Tools and Environment.....	60
Environment.....	60
Tools	60
4.3 Test Cases	61

4.3.1 Test Case: Student Registration (UC1)	61
S4.3.2 Test Case: Student Login (UC2)	61
4.3.3 Test Case: Ask Academic Query (UC3)	61
4.3.4 Test Case: View Chatbot Response (UC4)	62
4.3.5 Test Case: View Query History (UC5)	62
4.3.6 Test Case: Admin Login (UC6)	62
4.3.7 Test Case: Admin Update Academic Data (UC7)	63
4.3.8 Test Case: Search Course by Name (UC8)	63
4.3.9 Test Case: Ask Policy / Fee / Timetable Information (UC9)	63
4.3.10 Test Case: Bilingual Query Input (UC10)	64
4.3.11 Test Case: Manage Course Information (UC11)	64
4.3.12 Test Case: Manage Fee Structure (UC12)	64
4.3.13 Test Case: Manage Timetables (UC13)	64
4.3.14 Test Case: Manage Policies & Guidelines (UC14)	65
4.3.15 Test Case: View Student Query Logs (UC15)	65
4.3.16 Test Case: Manage User Accounts (UC16)	65
4.3.17 Test Case: Generate Reports (UC17)	65
4.3.18 Test Case: Admin Logout (UC18)	66
4.4 Summary	66
CHAPTER: 05	67
CONCLUSION	67
5.1 Conclusion	67
References	68

List of Figures

• Figure 1.1: System Flow	11
• Figure 1.2: Project Schedule (Gantt Chart)	13
• Figure 2.1: Use Case Diagram	20
• Figure 2.2: System Sequence Diagram – Student Registration	28
• Figure 2.3: System Sequence Diagram – Student Login	28
• Figure 2.4: System Sequence Diagram – Ask Academic Query	29
• Figure 2.5: System Sequence Diagram – View Chatbot Response.....	29
• Figure 2.6: System Sequence Diagram – View Query History	30
• Figure 2.7: System Sequence Diagram – Admin Login	30
• Figure 2.8: System Sequence Diagram – Admin Update Academic Data	31
• Figure 2.9: System Sequence Diagram – Search Course by Name	31
• Figure 2.10: System Sequence Diagram Ask Policy / Fee / Timetable Information..	32
• Figure 2.11: System Sequence Diagram – Bilingual Query Input	32
• Figure 2.12: System Sequence Diagram – Manage Course Information.	33
• Figure 2.13: System Sequence Diagram – Manage Fee Structure	33
• Figure 2.14: System Sequence Diagram – Manage Timetables	34
• Figure 2.15: System Sequence Diagram – Manage Policies & Guidelines	34
• Figure 2.16: System Sequence Diagram – View Student Query Logs	35
• Figure 2.17: System Sequence Diagram – Manage User Accounts	35
• Figure 2.18: System Sequence Diagram – Generate Reports	36
• Figure 2.19: System Sequence Diagram – Admin Logout	36
• Figure 2.20: Sequence Diagram – Student Registration	37
• Figure 2.21: Sequence Diagram – Student Login	37
• Figure 2.22: Sequence Diagram – Ask Academic Query	38
• Figure 2.23: Sequence Diagram – View Chatbot Response.....	38
• Figure 2.24: Sequence Diagram – View Query History	39
• Figure 2.25: Sequence Diagram – Admin Login	39
• Figure 2.26: Sequence Diagram – Admin Update Academic Data	40
• Figure 2.27: Sequence Diagram – Search Course by Name	40
• Figure 2.28: Sequence Diagram – Ask Policy / Fee / Timetable Information.....	41
• Figure 2.29: Sequence Diagram – Bilingual Query Input	41
• Figure 2.30: Sequence Diagram – Manage Course Information.	42
• Figure 2.31: Sequence Diagram – Manage Fee Structure	42
• Figure 2.32: Sequence Diagram – Manage Timetables	43
• Figure 2.33: Sequence Diagram – Manage Policies & Guidelines.....	43
• Figure 2.34: Sequence Diagram – View Student Query Logs	44
• Figure 2.35: Sequence Diagram – Manage User Accounts	44
• Figure 2.36: Sequence Diagram – Generate Reports	45
• Figure 2.37: Sequence Diagram – Admin Logout	45
• Figure 3.1: Architecture Diagram of AI-Powered Chat-Based Academic Advisor .	51
• Figure 3.2: Class Diagram of AI-Powered Chat-Based Academic Advisor.....	52
• Figure 3.3: Combined Activity Diagram	54

List of Tables

• Table 1.1: Risk & Contingencies	12
• Table 1.2: Comparative Analysis of Existing Local Systems	14
• Table 2.1: UC1 – Student Registration	21
• Table 2.2: UC2 – Student Login	21
• Table 2.3: UC3 – Ask Academic Query	21
• Table 2.4: UC4 – View Chatbot Response	22
• Table 2.5: UC5 – View Query History	22
• Table 2.6: UC6 – Admin Login	22
• Table 2.7: UC7 – Admin Update Academic Data	23
• Table 2.8: UC8 – Search Course by Name	23
• Table 2.9: UC9 – Academic Policy Query	23
• Table 2.10: UC10 – Bilingual Query Input	23
• Table 2.11: UC11 – Manage Course Information	24
• Table 2.12: UC12 – Manage Fee Structure	24
• Table 2.13: UC13 – Manage Timetables	24
• Table 2.14: UC14 – Manage Policies	25
• Table 2.15: UC15 – View Query Logs	25
• Table 2.16: UC16 – Manage User Accounts	25
• Table 2.17: UC17 – Generate Reports	26
• Table 2.18: UC18 – Admin Logout	26
• Table 3.1: Requirements Traceability Matrix (RTM)	47
• Table 3.2: Component Description	53

CHAPTER - 01

SOFTWARE PROJECT MANAGEMENT PLAN

1.1-Introduction

The rising need to access academic support in a timely manner has put forth the necessity of having an effective communication in university. Traditional academic advising is constrained by office hours, the number of personnel available and the increase in students in need of help. Students will usually have difficulties in getting reliable answers to the academic questions fast.

This is a process that may take up much time since they have to go through different resources such as course catalogs, academic regulations and departmental guidelines. In that regard, the given project will attempt to develop a web-based application within the Computer Science Department in Quaid-i-Azam University (QAU).

With this system, the students will have direct access to academic information, by querying a structured body of knowledge, consisting of course outlines, academic policies, and outlining fee policies. This is aimed at cutting down on administrative burden, enhancing student experience and offering academic guidance to students in a fast and precise way.

1.2-Problem Definition

Students in the Computer Science Department at Quaid-i-Azam University (QAU) have a lot of trouble getting academic information, especially about academic matters, this information is so scattered across different places like department offices, official university websites, and academic advisors, which makes it hard for students to find what they need. Offices are only open for hours, so students can't always get help when they need it. Advisors are also very busy, so students don't always get answers quickly. Many students end up asking the same questions over and over, which makes the staff's work even harder. On top of all this, academic documents like course catalogs and rules are very long and formal, which makes it difficult for students to find the specific information they are looking for.

.

1.3 Proposed System

Our project will act as an academic advisor for Computer Science students at Quaid-i-Azam University. Main goal is to provide students with fast and easy access to academic information through a simple chat interface. Instead of going to the department office, students can simply ask

questions in natural language and get immediate answers. The system will allow students to ask academic questions like What are the prerequisites for Machine Learning?, search for course details such as Data Structure.

1. Dataset

The dataset for our project will include information like course details such as titles, descriptions, and credit hours prerequisite courses, fee structure and payment guidelines, class schedules and examination timetables, academic policies and departmental rules, as well as final year project and thesis submission guidelines. The information is stored in a database, allowing the system to quickly retrieve relevant details when a student asks a question.

2. Preprocessing

These steps include:

- Tokenization which means Splitting the sentence into individual words
- Text Cleaning which means removing unnecessary symbols and characters
- Stop-word Removal that is eliminating common words that do not affect meaning
- Normalization which converting the query into a standard format

These are the steps that help the system to understand the questions more clearly.

3. Pretrained Model

The system uses a **Multilingual BERT model** to interpret user queries. The reason why this model is selected is because it is able to comprehend context in sentences, and also it is able to accommodate several languages such as English and Urdu. The fact that a pretrained model is used also minimizes the time required to train a complex model because still good results are obtained.

4. Intent Detection and Information Retrieval

Following are the steps system followed

- Detects the purpose of the query for example course prerequisites or fee information
- Check terms such as course names
- Search the database for matching records
- Convert the retrieved information into easy and understandable way for the student

5. Deployment

The system will be deployed as a **web-based chatbot application** that students can access through their browsers on computers or mobile devices through which user can:

- Ask academic questions
- Receive quick responses about courses, prerequisites, or policies
- Access important academic information at any time

The system will also have administrator panel where authorized personals can update information and schedules and policies.

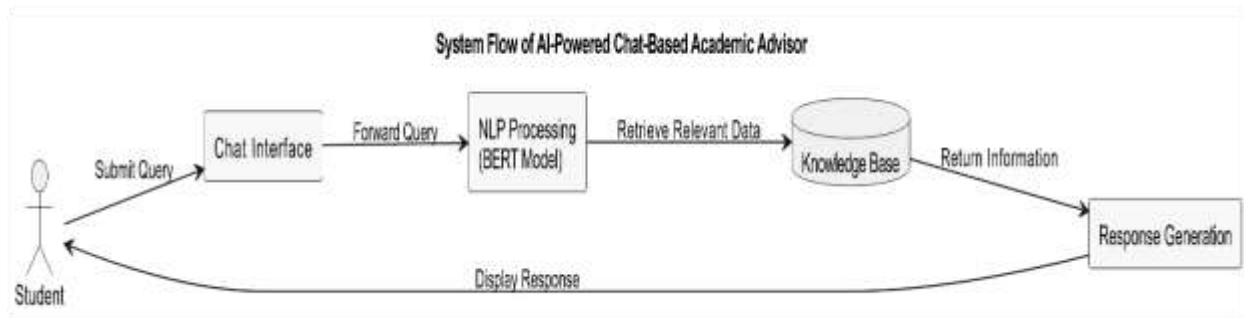


Figure 1.1 System Flow

1.4 Scope

Our system will provide information related to :

- Registration of courses offered in department/
- Course prerequisites and eligibility requirements.
- Credit hour distribution across semesters.
- Fee structure and payment guidelines.
- Class schedules and examination timetables.
- Final year project and thesis guidelines
- Departmental academic policies and regulations

The system will be available through the website of Quaid I Azam university Computer Science department website **interface**, allowing students to access it from a desktop or mobile browser. It is a bilingual supported system it will supports questions in both Urdu and English language in which student is more comfortable.

The project will not replace ai advisor because it cannot make official academic decisions the purpose is to assist students and reduce workload on staff.

1.5 Objectives

User Objectives

Our system will help students by providing instant answers to their academic questions. It will assist them in understanding course prerequisites, credit hours, fee structure, and academic policies in both English and Urdu. This will make it easier for students to get the information they need and reduce confusion about academic procedures and semester planning.

Technical Objectives

A Multilingual BERT trained model will be used in the system to comprehend and answer student queries. It will also be structured in terms of knowledge base details of the courses and policies. The chatbot will be implemented as a web platform and will be affordable to all devices.

Learning Objectives

This system aims to assist students in learning to process natural language queries and comprehending the working of NLP models such as BERT, as well as, give them a practical aid in academic support.

1.6 Motivation

- Students may frequently be interested in courses, prerequisites and academic policies.
- Sometimes the advisors in the departments and academic university may not be present and accessible.
- Sometimes there is wasting of time by students in searching the notices, regulations and course outlines.
- Because of this, a number of students pose same academic questions to the academic again and again; this overburdens the administrative members of staff.
- Chatbot can be an AI-based service representing a fast and trustworthy response to frequent academic questions.
- Intelligent assistants are being exploited in the educational systems, and these types of solutions are becoming helpful and pertinent.

1.7 Risk & Contingencies

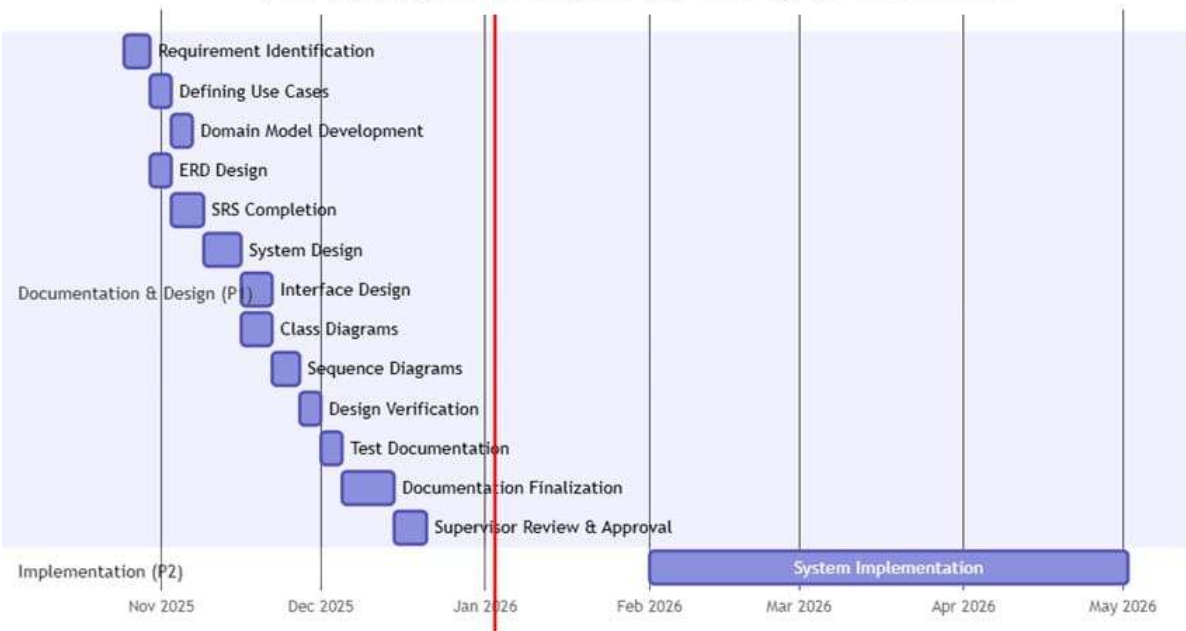
Risk	Impact	Mitigation
Incomplete academic data	Medium	Organize official data into a structured knowledge base
Incorrect query interpretation	High	Fine-tune BERT model on academic queries and test regularly
Expired polices or expired schedules	High	Add admin panel for updating information
Limited performance	Medium	Use training data in English and Urdu
System integration issues	Medium	Perform regular testing and development
Low student adoption	Medium	Design a simple and user-friendly interface

Table 1.1: Risk & Contingencies

Project Schedule

		Name	Duration	Start	Finish	Predecessors	Resource Names
1	P	Requirement Identification	5 days	25-Oct-2025	29-Oct-2025		Usman Mustafa, MS Word, Requirement Notes
2	T	Defining Use Cases	4 days	30-Oct-2025	02-Nov-2025	1	Usman Mustafa, Draw.io, StarUML, Laptop
3	M	Develop Domain Model	4 days	03-Nov-2025	06-Nov-2025	2	Usman Mustafa, Draw.io, Domain Knowledge, Laptop
4	T	ERD Design	4 days	07-Nov-2025	10-Nov-2025	1	Usman Mustafa, Draw.io, MySQL Workbench, Laptop
5	T	Software Requirements Specification (SRS)	6 days	13-Nov-2025	18-Nov-2025	1,2,4	Usman Mustafa, Laptop, MS Word, IEEE SRS Template
6	T	System Design Development	7 days	17-Nov-2025	23-Nov-2025	5	Usman Mustafa, Draw.io, StarUML, Laptop, Design Notes
7	T	Develop Interfaces	6 days	24-Nov-2025	29-Nov-2025	6	Usman Mustafa, Figma, HTML/CSS, React, VS Code
8	T	Class Diagram Designing	6 days	30-Nov-2025	05-Dec-2025	6	Usman Mustafa, Draw.io, StarUML, Laptop
9	T	Sequence Diagrams	5 days	06-Dec-2025	10-Dec-2025	8	Usman Mustafa, PlantUML, Draw.io, Laptop
10	T	Design Phase Verification	4 days	11-Dec-2025	14-Dec-2025	7,8,9	Usman Mustafa, Review Checklist, Laptop
11	T	Software Test Documentation	4 days	15-Dec-2025	18-Dec-2025	10	Usman Mustafa, Laptop, MS Word, Test Case Sheets
12	T	Documentation Finalization (P1)	10 days	19-Dec-2025	28-Dec-2025	11	Usman Mustafa, Laptop, MS Word, Grammarly
13	M	Supervisor Review & Approval	6 days	29-Dec-2025	03-Jan-2026	12	Supervisor, Usman Mustafa, Final Report Draft
14	P	System Implementation Phase (P2)	3 months	01-Feb-2026	30-Apr-2026	13	nan

Ai Academic advisor for CS QAU Schedule



1.2: Project Schedule

1.8 Application

- Academic query processing on Computer Science students.
- Course prerequisite and credit-hour information.
- Semester planning support
- Fee structure and timetable information.
- Thesis and research guideline.
- Reducing the repetitive workload of student advisor.
- 24/7 academic information access to students.

1.9 Comparative Analysis of Existing Local Systems

- Previous systems are **static** and **not interactive**.
- Academic **advisors** are available only during limited hours.
- These systems **don't support intelligent bilingual question answering**.
- Some international universities also have chatbots but they don't support multilingual they are only limited to English and are based on their local rules

Platform	Chat-Based	English/Urdu Support	Department-Specific	AI-Based
Department Website	No	Partial	Yes	No
Printed Handbook	No	No	Yes	No
Academic Advisor (Manual)	Yes	Yes	Yes	No
Generic University Portal	No	Partial	Partial	No
Proposed System	Yes	Yes	Yes	Yes

Table 1.2: Comparative Analysis of Existing Local Systems

1.10 Project Organization

Process Model

The development of AI-powered chatbot is being managed with the assistance of the Agile Scrum model arranged in small, manageable steps. The main activities that are done during each sprint include preparation of the knowledge base, integration of the NLP model, development of the chatbot interface and testing/ debugging. The project's team roles will consist of a Project Manager who organizes tasks, planning, and schedule, an AI Engineer who will compile and customize the BERT model to process queries, a Backend Developer who will create APIs, database connectivity, system logic, a Frontend Developer who will create the chatbot user

interface, and a Tester who will verify the responses of the chatbot and make him usable. **Tools & Technologies**

- Python
- TensorFlow / PyTorch
- Multilingual BERT
- Google Colab / Kaggle for model experimentation
- Flask / FastAPI / Django for backend development
- MySQL / PostgreSQL for structured academic database
- HTML, CSS, JavaScript or React for frontend development

.

CHAPTER – 02

SOFTWARE REQUIREMENTS SPECIFICATION

2.1 Introduction

The software requirements for our project are covered in this chapter. Questions regarding **prerequisites, credit hours, fee structures, class schedules, exam schedules, and thesis guidelines**, Our system offers students immediate academic support.

Our system can comprehend and answer questions in both Urdu and English thanks to artificial **intelligence (AI) and natural language processing (NLP)**, because of which it became possible for making our system and reducing the load of academic advisor.

2.2 Product Overview

The **AI-Powered Chat-Based Academic Advisor** is designed to serve the stakeholders such as:

Students:

Who need guidance in courses, prerequisites, fee structure, timetables, thesis requirements, and departmental policies.

Newly Admitted Students:

Students who are unfamiliar with departmental procedures or want to take admission in Qau can have access to academic information.

Academic Advisors:

cannot replace academic advisor but will be very helpful in reducing the workload on academic advisor or staff members.

Departmental Administration:

staff can use the system to streamline academic communication and keep information organized.

2.3 Specific Requirements

Functional and Technical Requirements:

1. **Text Query Input:**

e.g. what are pre req of machine learning

2. **Course Search:**
e.g. details about data structures
3. **Policy & Info Lookup:**
e.g. what are the rules for submitting FYP
4. **Bilingual Support:**
e.g. who is teaching CS243
e.g CS243 kon parha rha?
5. **Intent Detection:**
what is the last date for submission of fees in spring 26?
6. **Knowledge Base Retrieval:**
when is the class of cs243 on Tuesday?
7. **Response Generation:**
System will generate clear answers.
8. **Admin Update Facility:**
Admins will update policies courses and information's related to the department.
9. **Student Query History:**
the history of all the questioners will be maintained.
10. **Web Accessibility:**
Ai advisor will be accessible on mobiles and desktop.

2.4 Software Product Features

- **Academic Query Input:** The students are allowed to pose questions regarding courses, prerequisites, credit hours, fees and policies.
- **Courses Information Search:** Students can search courses details by name.
- **Bilingual Support:** Our project will understands both **English** and **Urdu** queries.
- **Knowledge Base Retrieval:** Our system will fetch the required data from database.
- **Conversational Responses:** Our system provides easily understandable answers in a chat format.
- **Student Query History:** Students can check past queries and responses.
- **Admin Panel:** Admins can update courses, policies, schedules, and other academic info.

2.5 Software System Attributes

The system must satisfy the following quality attributes:

- **Usability:**

The interface must be simple, clean, and easy to use for students with technical backgrounds.

- **Performance:**

The system should return responses quickly.

- **Reliability:**

The advisor must be consistent in answers and provide correct and relevant academic response based on data provided .

- **Scalability:**

The system should support future expansion and additional services.

- **Security:**

Data should be secure through proper authentication and controlled access.

- **Maintainability:**

The system must be easy to update when the admin wants to change policies and information.

2.6 List of Use Cases

- **UC1:** Student Registration
- **UC2:** Student Login
- **UC3:** Ask Academic Query
- **UC4:** View Chatbot Response
- **UC5:** View Query History
- **UC6:** Admin Login
- **UC7:** Admin Update Academic Data
- **UC8:** Search Course by Name
- **UC9:** Ask Policy / Fee / Timetable Information
- **UC10:** Bilingual Query Input (English / Urdu)

2.7 Use Case Diagram

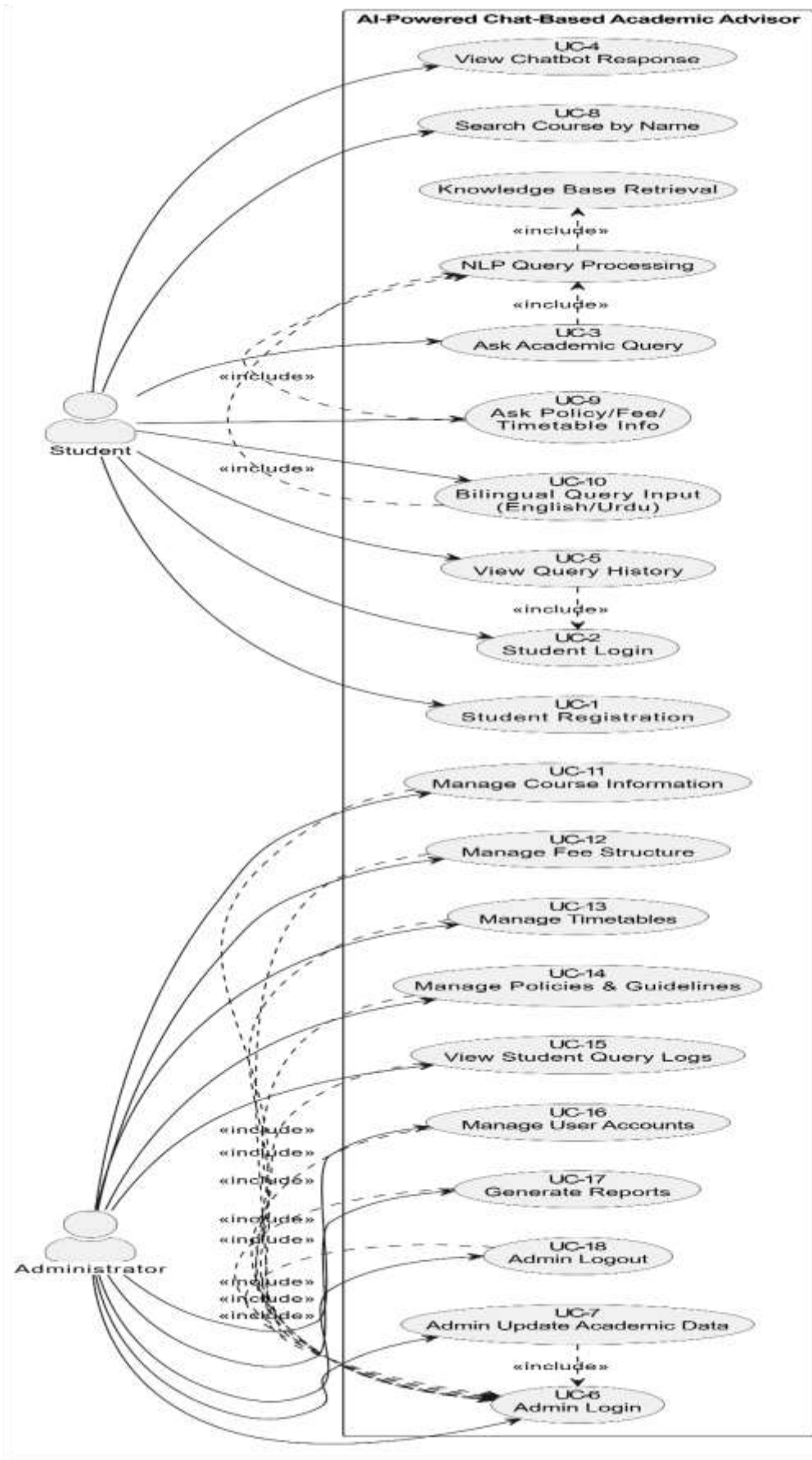
There are two primary actors admin and students and the interaction is shown by the use case diagram below:

The Use case diagram is divided into two logical parts

- **Student Interaction (UC1, UC2, UC3, UC4, UC5, UC8, UC9, UC10)** allows students to register, log in, ask queries, view responses, search courses, and check history.
- **Admin Oversight (UC6, UC7, UC11, UC12, UC13, UC14, UC15, UC16, UC17, UC18)** enables administrators to log in , update academic records, manage course information, manage

fees structure, , manage timetables , manage policies & guidelines , view student query logs ,manage user accounts , generate reports and admin logout.

Figure 2.1: Use Case Diagram



2.7.1 Use Case Descriptions

Use Case 1: Student Registration

Use Case	UC1: Student Registration
Primary Actor	Student
Pre-Conditions	Student is not registered in the system
Post-Conditions	Student account is created and stored in the database
Main Scenario	1. Student opens the system and selects “Register”. 2. System displays registration form. 3. Student enters required details. 4. System validates the data. 5. System creates account and confirms registration.
Alternative Scenario	5a. If email already exists, system shows an error. 5b. If required fields are missing, system asks student to complete the form.

Table 2.1: UC1 – Student Registration

Use Case 2: Student Login

Use Case	UC2: Student Login
Primary Actor	Student
Pre-Conditions	Student is registered
Post-Conditions	Student is authenticated and redirected to home/chat screen
Main Scenario	1. Student selects “Login”. 2. System displays login form. 3. Student enters credentials. 4. System validates credentials. 5. On success, system grants access.
Alternative Scenario	4a. Invalid email/password results in an error message. 5a. Network/system issue results in login failure notice.

Table 2.2: UC2 – Student Login

Use Case 3: Ask Academic Query

Use Case	UC3: Ask Academic Query
Primary Actor	Student
Pre-Conditions	Student is logged in
Post-Conditions	Query is processed and relevant answer is generated
Main Scenario	1. Student opens chat interface. 2. Student types academic query. 3. System preprocesses query.

	4. NLP model detects intent. 5. Knowledge base is searched. 6. System generates response.
Alternative Scenario	3a. If query is unclear, system asks the student to rephrase. 5a. If no result is found, system informs the student accordingly.

Table 2.3: UC3 – Ask Academic Query

Use Case 4: View Chatbot Response

Use Case	UC4: View Chatbot Response
Primary Actor	Student
Pre-Conditions	Query has been processed
Post-Conditions	Student reads academic guidance returned by the chatbot
Main Scenario	1. System displays the answer. 2. Student reviews the response. 3. Student may ask follow-up queries.
Alternative Scenario	1a. If response confidence is low, system suggests rephrasing the question or contacting the department.

Table 2.4: UC4 – View Chatbot Response

Use Case 5: View Query History

Use Case	UC5: View Query History
Primary Actor	Student
Pre-Conditions	Student is logged in
Post-Conditions	Student views previous queries and responses
Main Scenario	1. Student opens “History”. 2. System retrieves stored queries. 3. System displays past queries and answers.
Alternative Scenario	1a. If no records exist, system shows “No history available”.

Table 2.5: UC5 – View Query History

Use Case 6: Admin Login

Use Case	UC6: Admin Login
Primary Actor	Administrator
Pre-Conditions	Admin account exists
Post-Conditions	Admin is authenticated and accesses dashboard
Main Scenario	1. Admin opens login page. 2. Admin enters credentials. 3. System validates credentials. 4. Dashboard access is granted.
Alternative Scenario	3a. Invalid credentials result in access denial message.

Table 2.6: UC6 – Admin Login

Use Case 7: Admin Update Academic Data

Use Case	UC7: Admin Update Academic Data
Primary Actor	Administrator
Pre-Conditions	Admin is logged in
Post-Conditions	Academic records are updated
Main Scenario	1. Admin opens dashboard. 2. Admin selects module (courses, fees, schedules). 3. Admin updates data. 4. System saves changes.
Alternative Scenario	3a. Invalid data results in validation error.

Table 2.7: UC7 – Admin Update Academic Data

Use Case 8: Search Course by Name

Use Case	UC8: Search Course by Name
Primary Actor	Student
Pre-Conditions	Student is logged in
Post-Conditions	Course information is displayed
Main Scenario	1. Student selects course search option. 2. Student enters course name. 3. System retrieves matching course. 4. Course details are displayed.
Alternative Scenario	3a If course is not found, system suggests alternative keywords.

Table 2.8: UC8 – Search Course by Name

Use Case 9: Ask Policy / Fee / Timetable Information

Use Case	UC9: Ask Policy / Fee / Timetable Information
Primary Actor	Student
Pre-Conditions	Student is logged in
Post-Conditions	Relevant academic information is displayed
Main Scenario	1. Student enters query about fees, policies, or timetable. 2. System detects query intent. 3. Knowledge base retrieves data. 4. System displays answer.
Alternative Scenario	3a .If no record exists, system informs student accordingly.

Table 2.9: UC9 – Academic Policy Query

Use Case 10: Bilingual Query Input

Use Case	UC10: Bilingual Query Input
Primary Actor	Student
Pre-Conditions	Student is logged in
Post-Conditions	System processes query in English or Urdu

Main Scenario	1. Student enters query in English or Urdu. 2. System preprocesses text. 3. BERT model interprets intent. 4. Knowledge base is queried. 5. System displays response.
Alternative Scenario	3a. If language is unclear, system asks student to rephrase query.

Table 2.10: UC10 – Bilingual Query Input

Use Case 11: Manage Course Information

Use Case	UC11: Manage Course Information
Primary Actor	Administrator
Pre-Conditions	Admin logged in
Post-Conditions	Course information updated
Main Scenario	1. Admin opens course management module. 2. Admin edits or adds course information. 3. System validates data. 4. System saves changes.
Alternative Scenario	3a. Invalid course data triggers validation error.

Table 2.11: UC11 – Manage Course Information

Use Case 12: Manage Fee Structure

Use Case	UC12: Manage Fee Structure
Primary Actor	Administrator
Pre-Conditions	Admin logged in
Post-Conditions	Fee records updated
Main Scenario	1. Admin opens fee module. 2. Admin edits or updates fee structure. 3. System saves changes.
Alternative Scenario	2a Invalid values trigger error message.

Table 2.12: UC12 – Manage Fee Structure

Use Case 13: Manage Timetables

Use Case	UC13: Manage Timetables
Primary Actor	Administrator
Pre-Conditions	Admin logged in
Post-Conditions	Updated timetable stored
Main Scenario	1. Admin opens timetable module. 2. Admin updates class schedule. 3. System validates and saves timetable.
Alternative Scenario	2a Incorrect schedule format results in validation error.

Table 2.13: UC13 – Manage Timetables

Use Case 14: Manage Policies & Guidelines

Use Case	UC14: Manage Policies
Primary Actor	Administrator
Pre-Conditions	Admin logged in
Post-Conditions	Policies updated
Main Scenario	1. Admin selects policy module. 2. Admin edits policy details. 3. System stores updated policy.
Alternative Scenario	2a, Missing information triggers validation message.

Table 2.14: UC14 – Manage Policies

Use Case 15: View Student Query Logs

Use Case	UC15: View Student Query Logs
Primary Actor	Administrator
Pre-Conditions	Admin logged in
Post-Conditions	Admin views student queries
Main Scenario	1. Admin opens query log module. 2. System retrieves logs. 3. Admin views query activity.
Alternative Scenario	2a If logs unavailable, system shows message.

Table 2.15: UC15 – View Query Logs

Use Case 16: Manage User Accounts

Use Case	UC16: Manage User Accounts
Primary Actor	Administrator
Pre-Conditions	Admin logged in
Post-Conditions	User accounts managed
Main Scenario	1. Admin views list of users. 2. Admin updates or disables account. 3. System saves changes.
Alternative Scenario	1a Unauthorized action triggers error.

Table 2.16: UC16 – Manage User Accounts

Use Case 17: Generate Reports

Use Case	UC17: Generate Reports
Primary Actor	Administrator

Pre-Conditions	Admin logged in
Post-Conditions	System generates usage reports
Main Scenario	1. Admin selects report module. 2. System processes data. 3. System displays report statistics.
Alternative Scenario	2a If insufficient data exists, system displays warning.

Table 2.17: UC17 – Generate Reports

Use Case 18: Admin Logout

Use Case	UC18: Admin Logout
Primary Actor	Administrator
Pre-Conditions	Admin logged in
Post-Conditions	Admin session terminated
Main Scenario	1. Admin selects logout option. 2. System ends session. 3. Admin redirected to login page.
Alternative Scenario	3a Session timeout logs admin out automatically.

Table 2.18: UC18 – Admin Logout

2.7.2 System Sequence Diagram

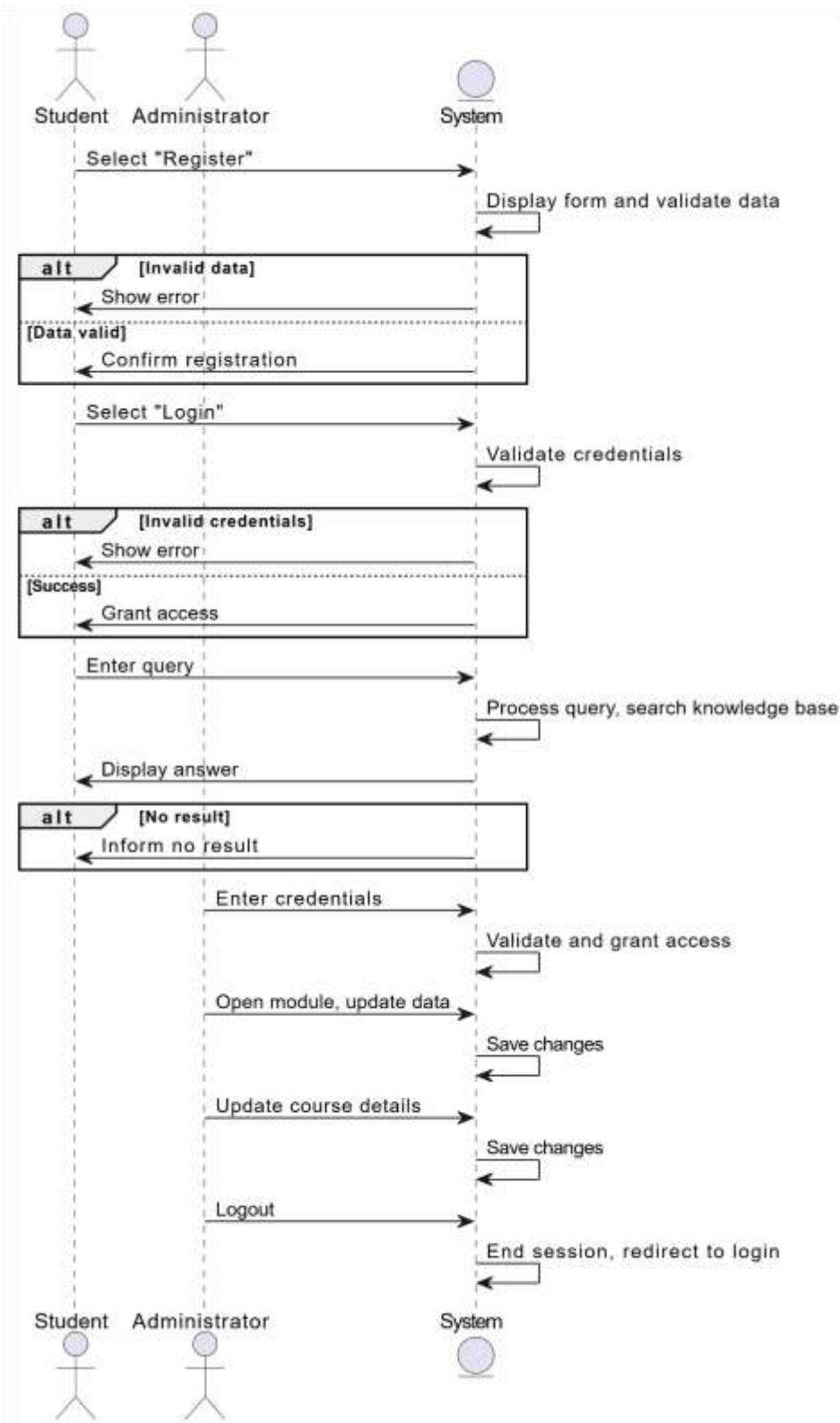


Figure 2.2: System Sequence Diagram – Student Registration

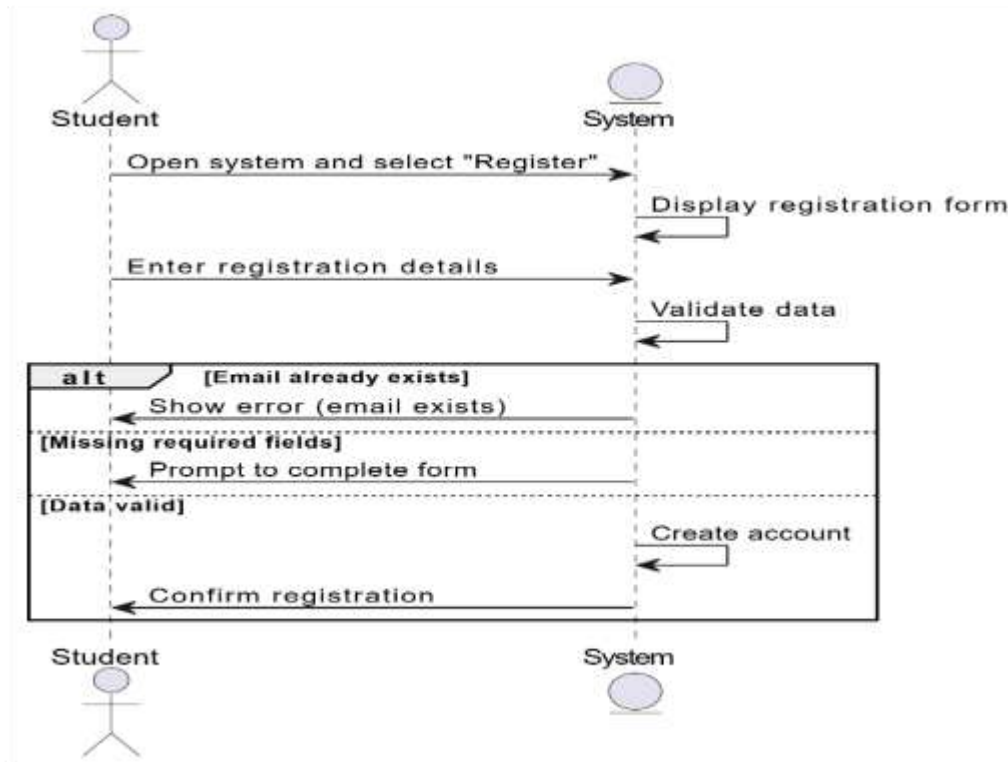


Figure 2.3: System Sequence Diagram – Student Login

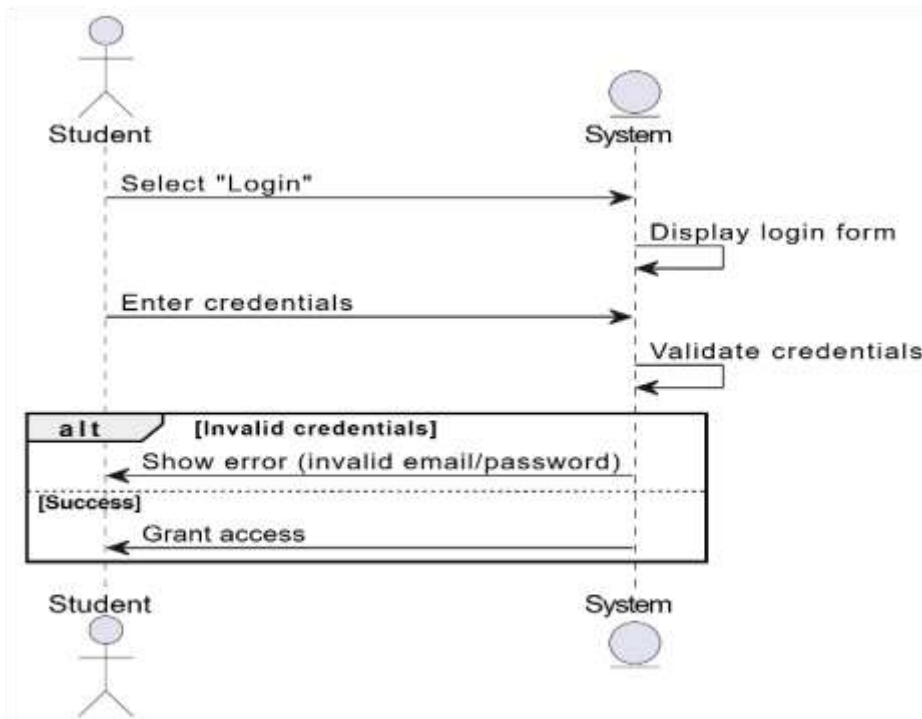


Figure 2.4: System Sequence Diagram – Ask Academic Query

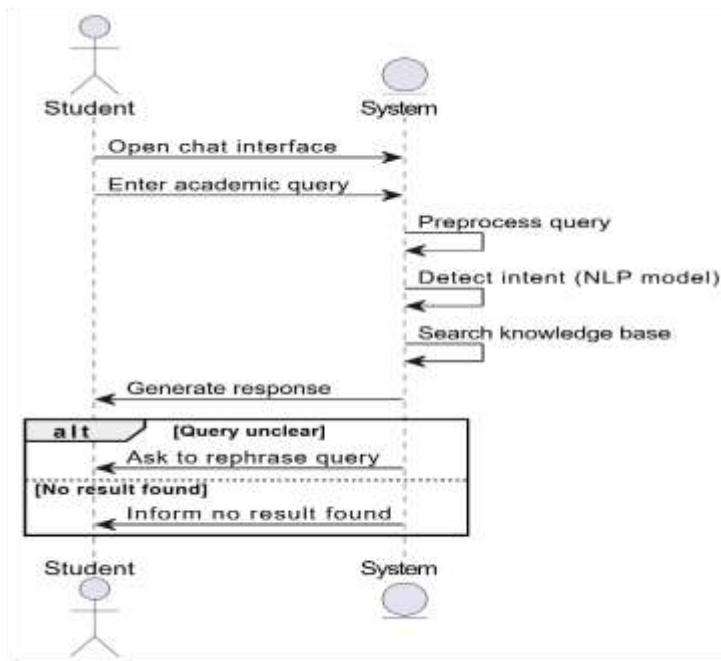


Figure 2.5: System Sequence Diagram – View Chatbot Response

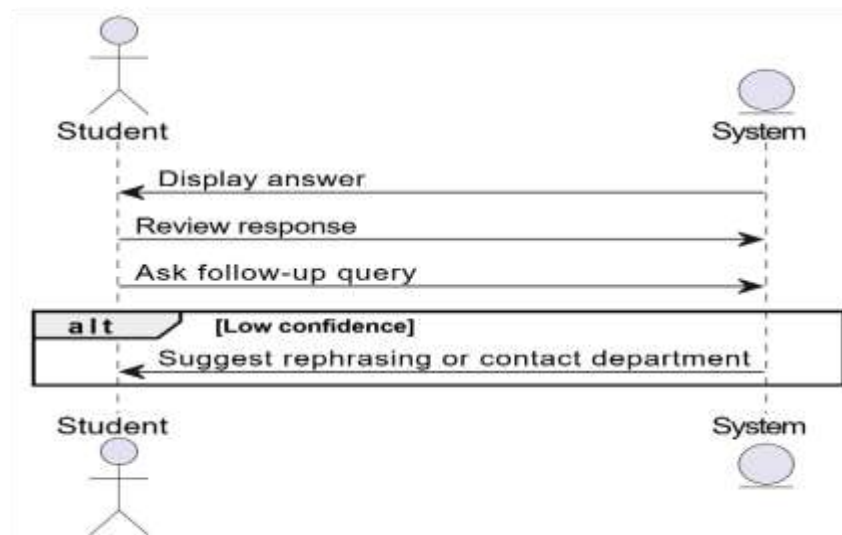


Figure 2.6: System Sequence Diagram – View Query History

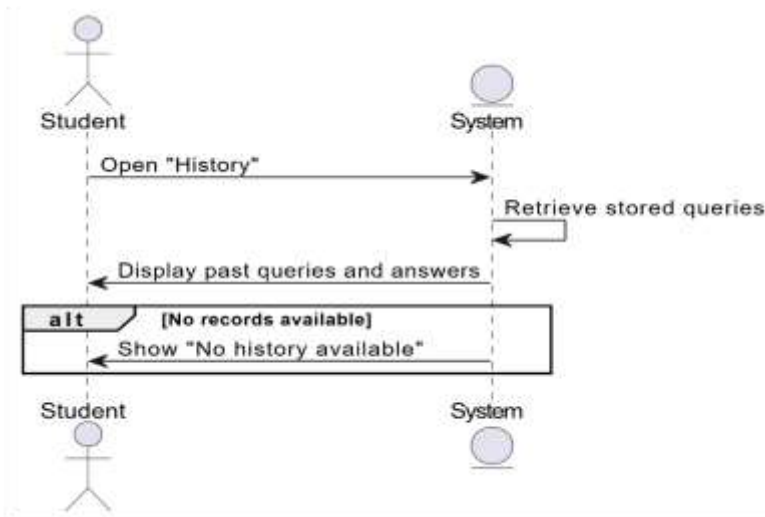


Figure 2.7: System Sequence Diagram – Admin Login

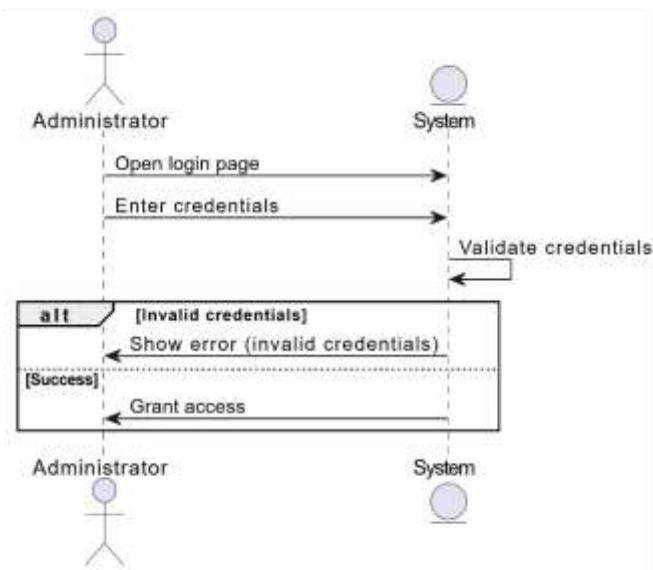


Figure 2.8: System Sequence Diagram – Admin Update Academic Data

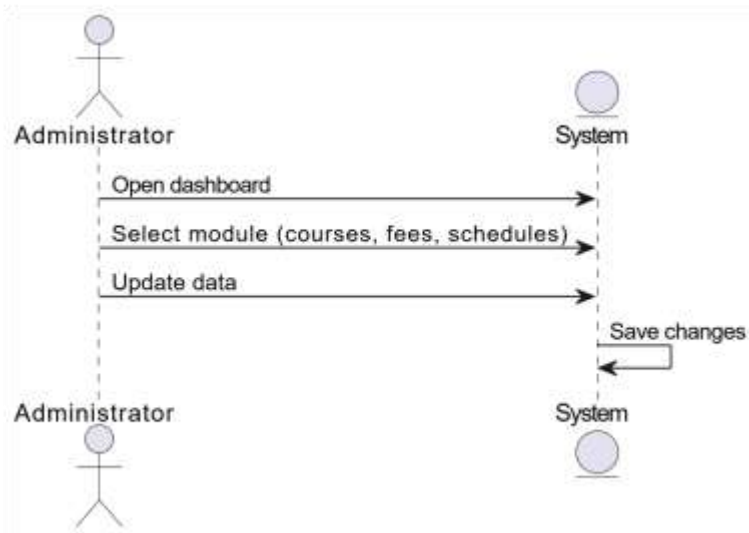


Figure 2.9: System Sequence Diagram – Search Course by Name

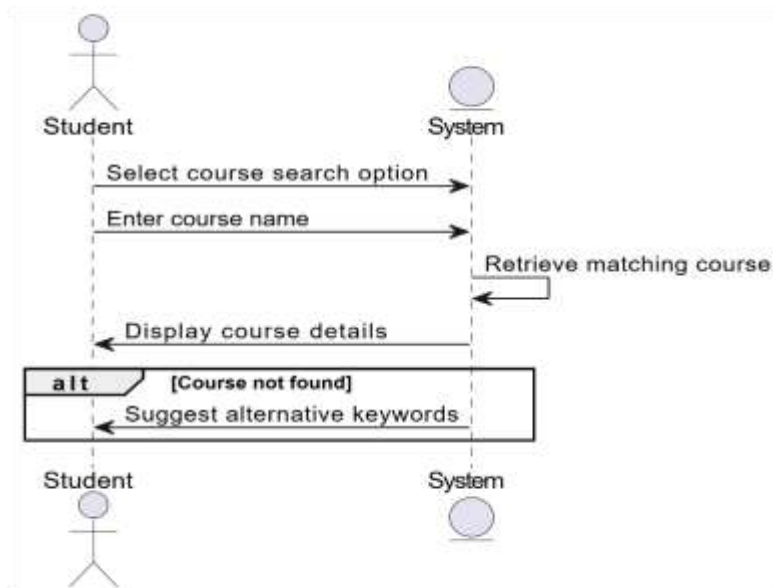


Figure 2.10: System Sequence Diagram – Ask Policy / Fee / Timetable Information

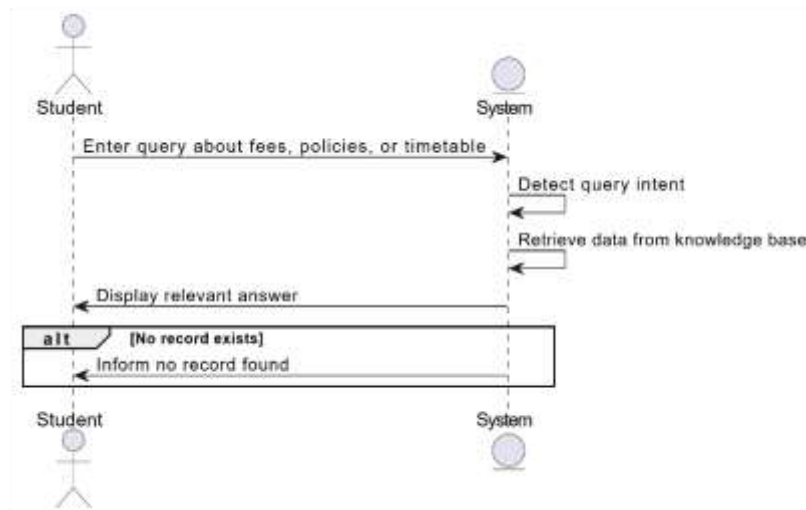


Figure 2.11: System Sequence Diagram – Bilingual Query Input

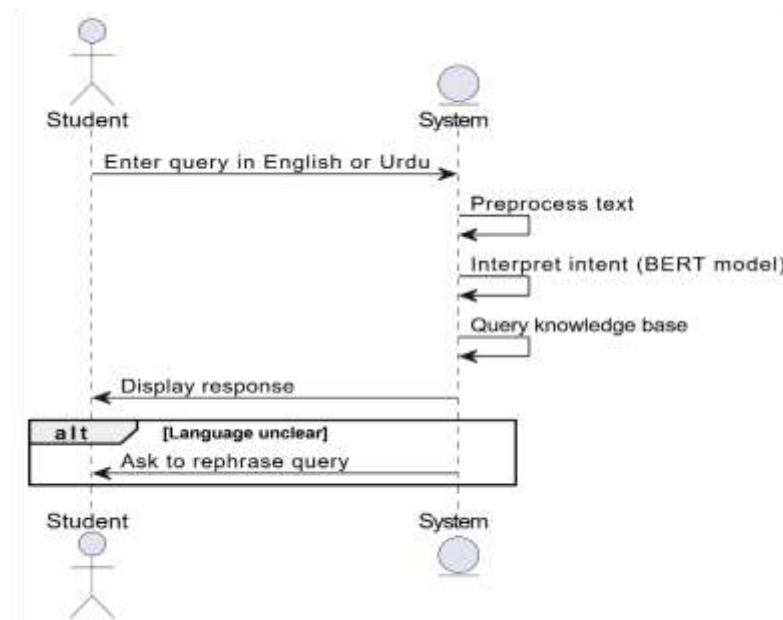


Figure 2.12: System Sequence Diagram – Manage Course Information

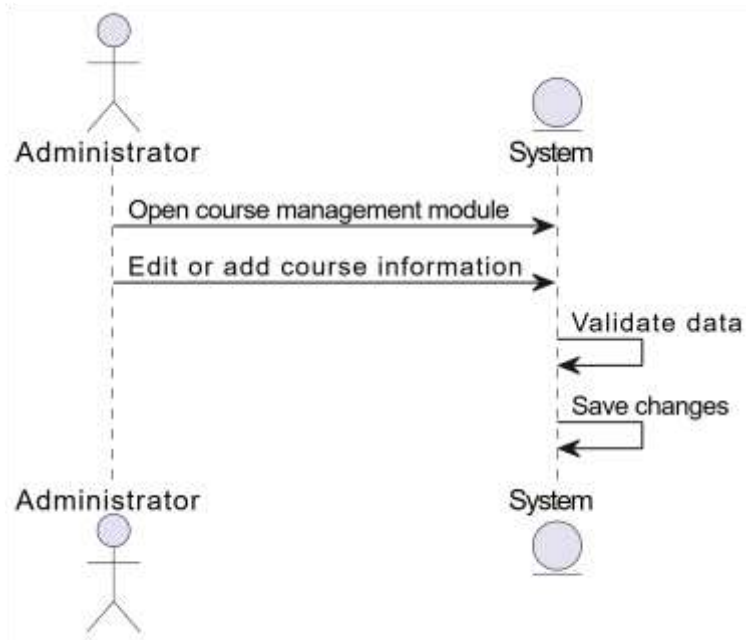


Figure 2.13: System Sequence Diagram – Manage Fee Structure

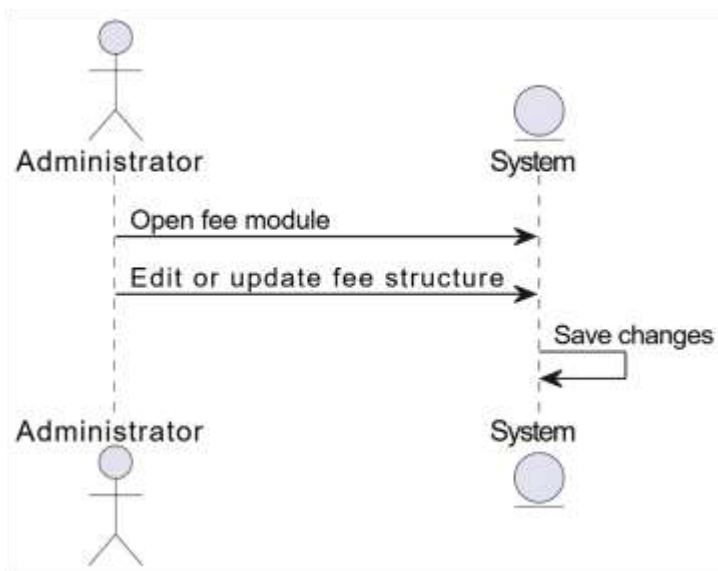


Figure 2.14: System Sequence Diagram – Manage Timetables

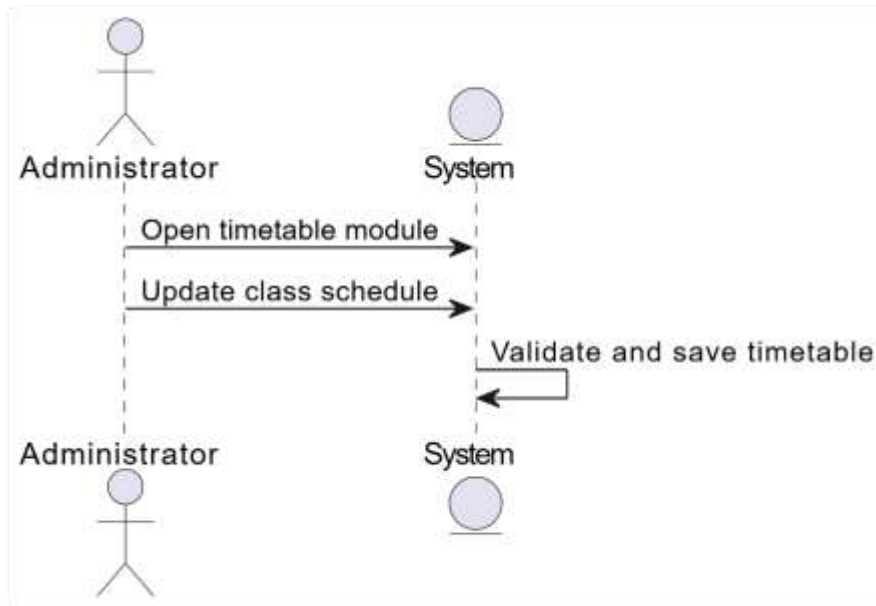


Figure 2.15: System Sequence Diagram – Manage Policies & Guidelines

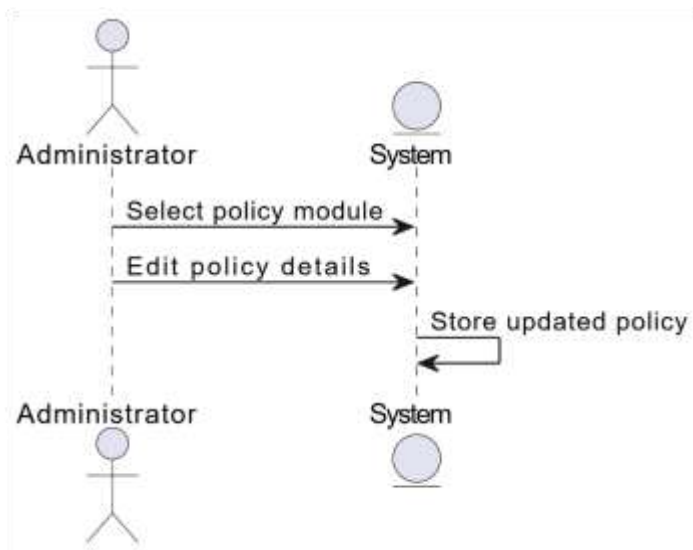


Figure 2.16: System Sequence Diagram – View Student Query Logs

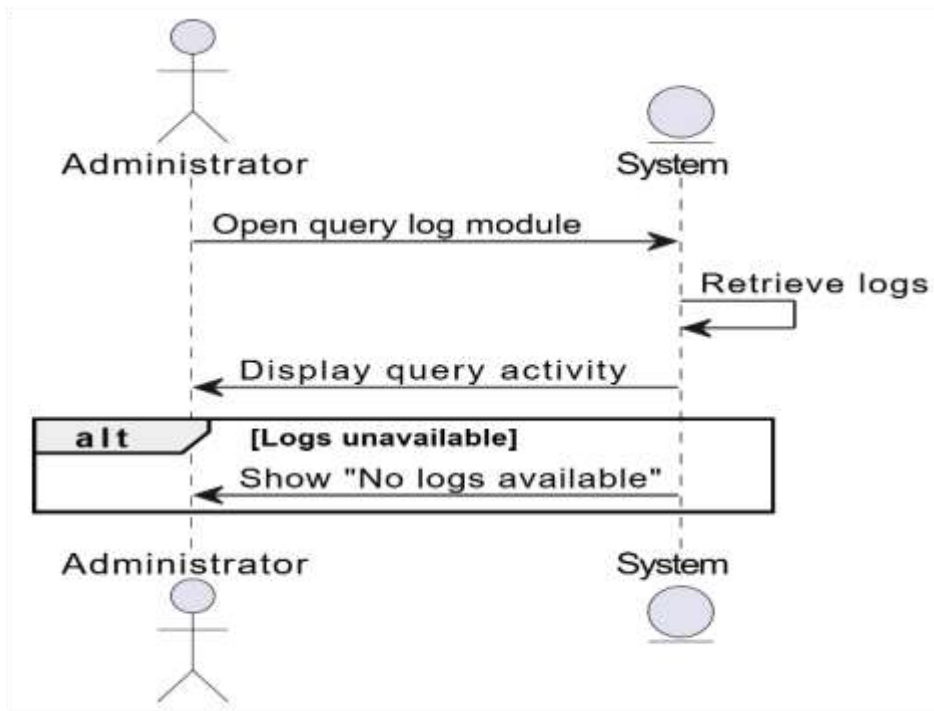


Figure 2.17: System Sequence Diagram – Manage User Accounts

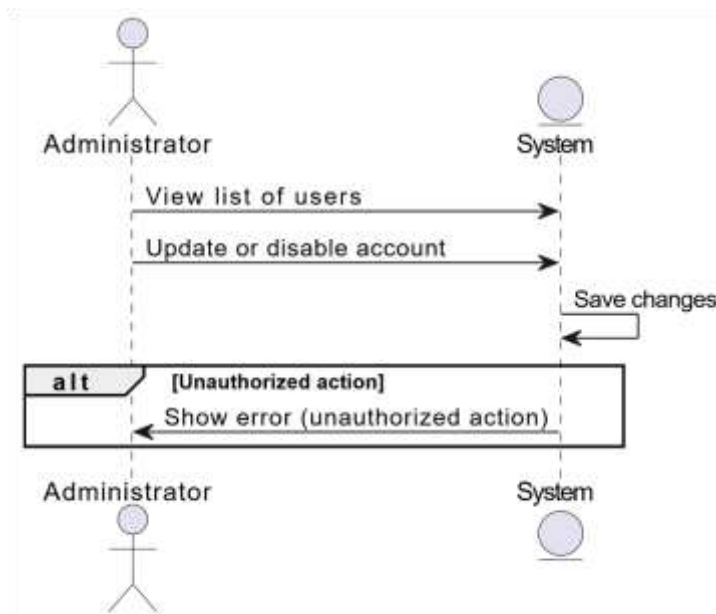


Figure 2.18: System Sequence Diagram – Generate Reports

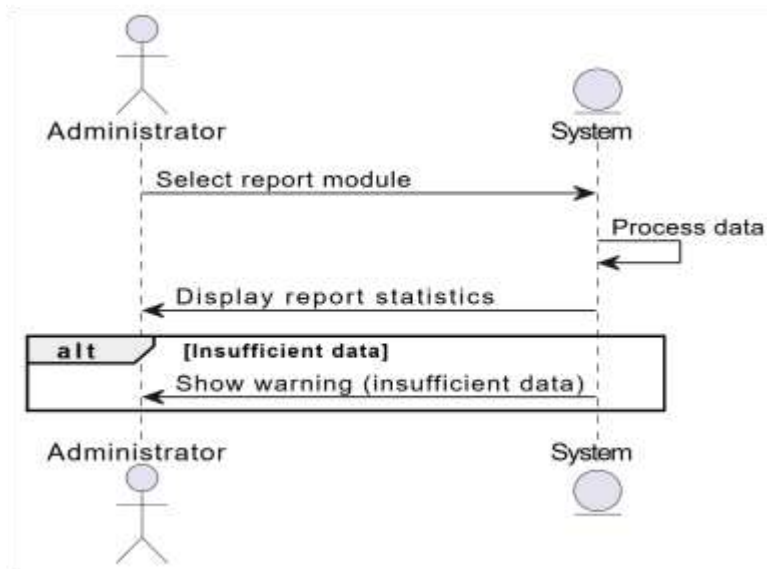
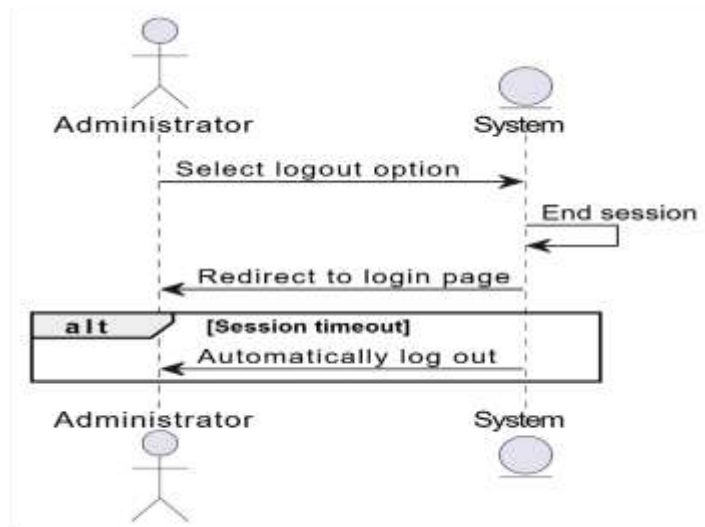


Figure 2.19: System Sequence Diagram – Admin Logout



2.7.3 Sequence Diagram

Figure 2.20: Sequence Diagram – Student Registration

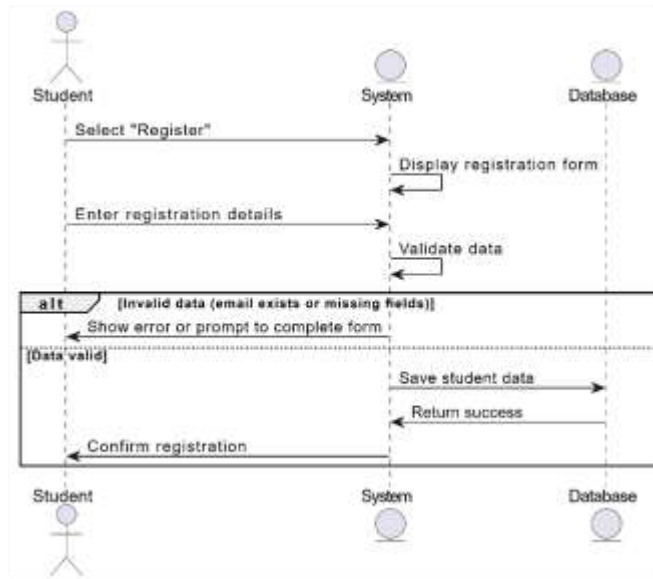


Figure 2.21: Sequence Diagram – Student Login

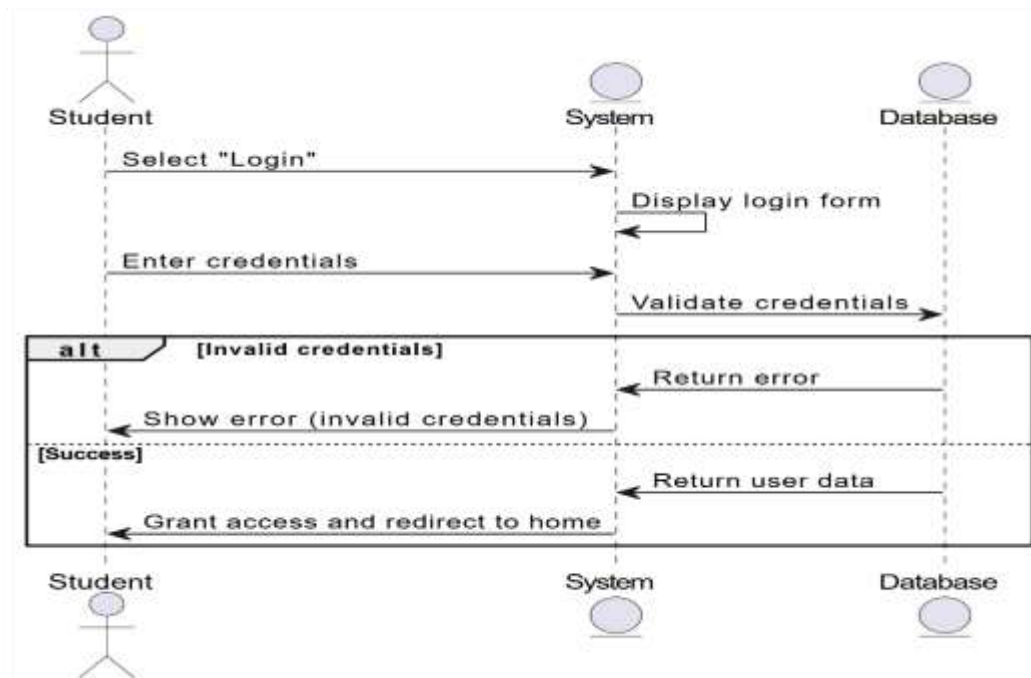


Figure 2.22: Sequence Diagram – Ask Academic Query

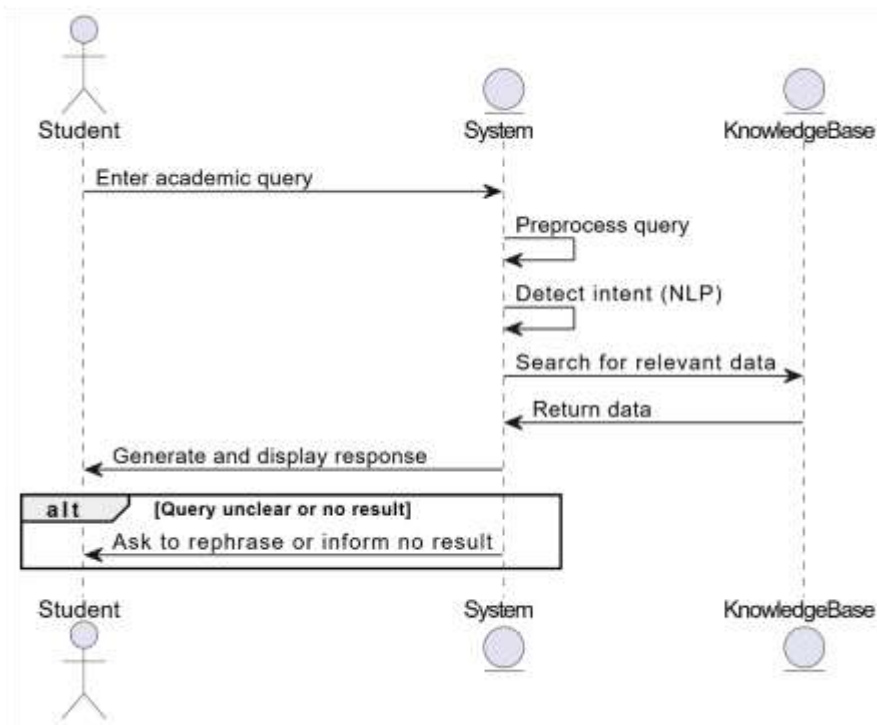


Figure 2.23: Sequence Diagram – View Chatbot Response

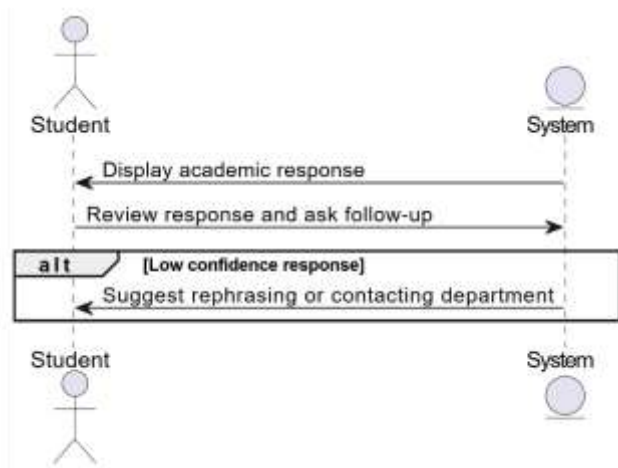


Figure 2.24: Sequence Diagram – View Query History

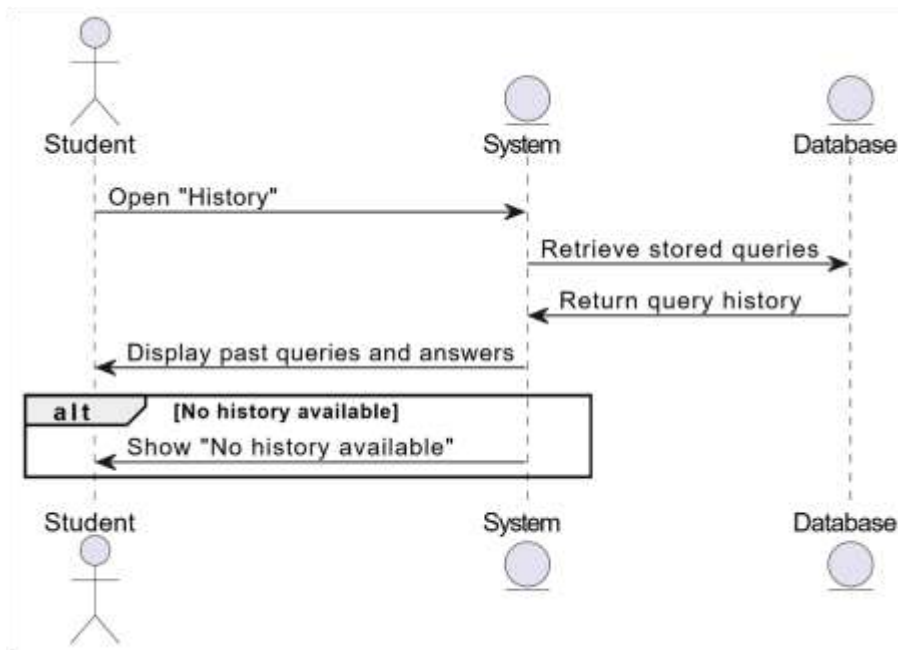


Figure 2.25: Sequence Diagram – Admin Login

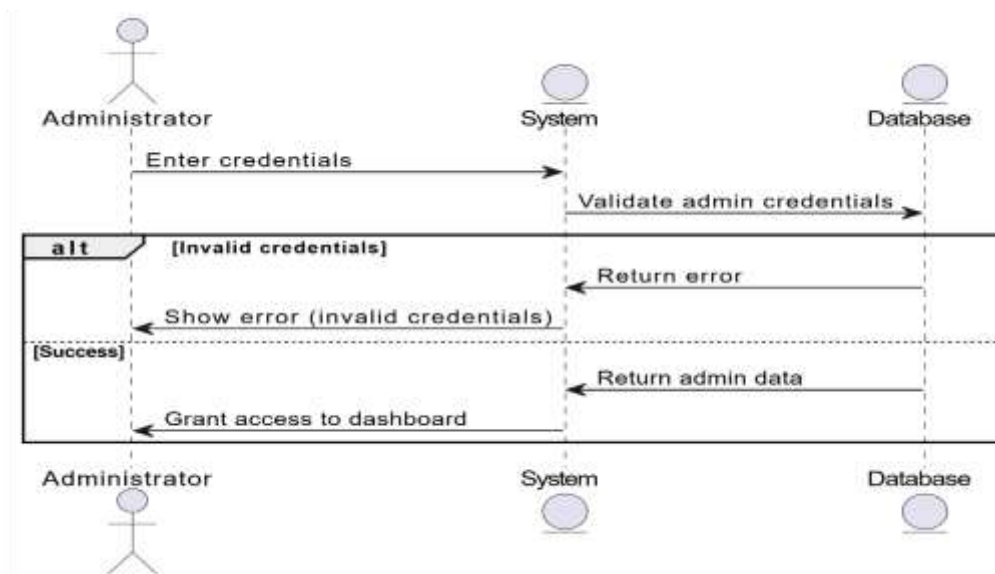


Figure 2.26: Sequence Diagram – Admin Update Academic Data

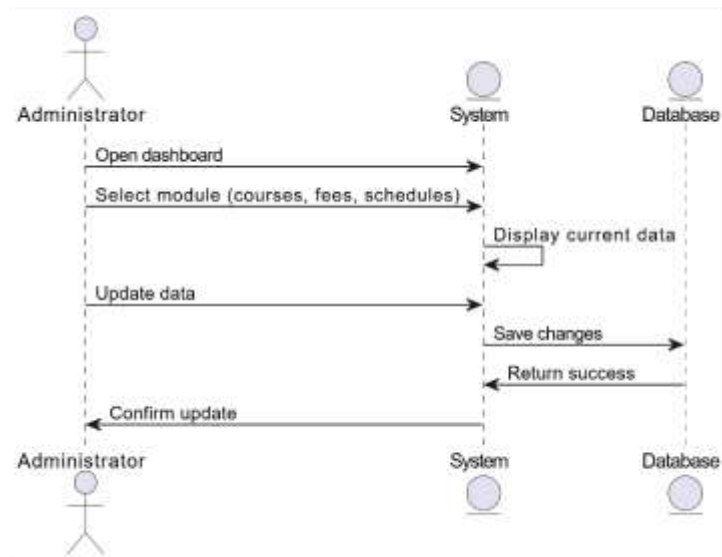


Figure 2.27: Sequence Diagram – Search Course by Name

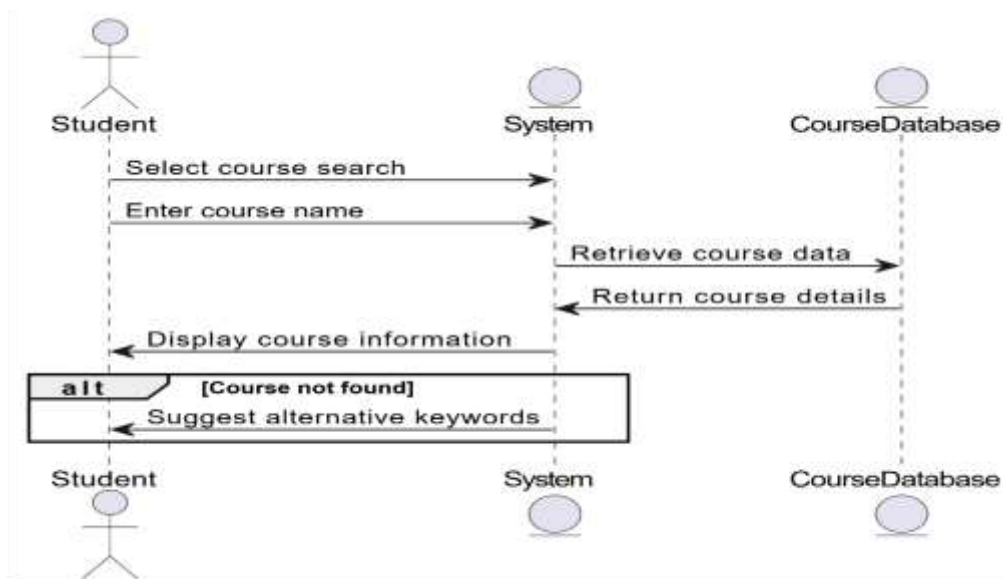


Figure 2.28: Sequence Diagram – Ask Policy / Fee / Timetable Information

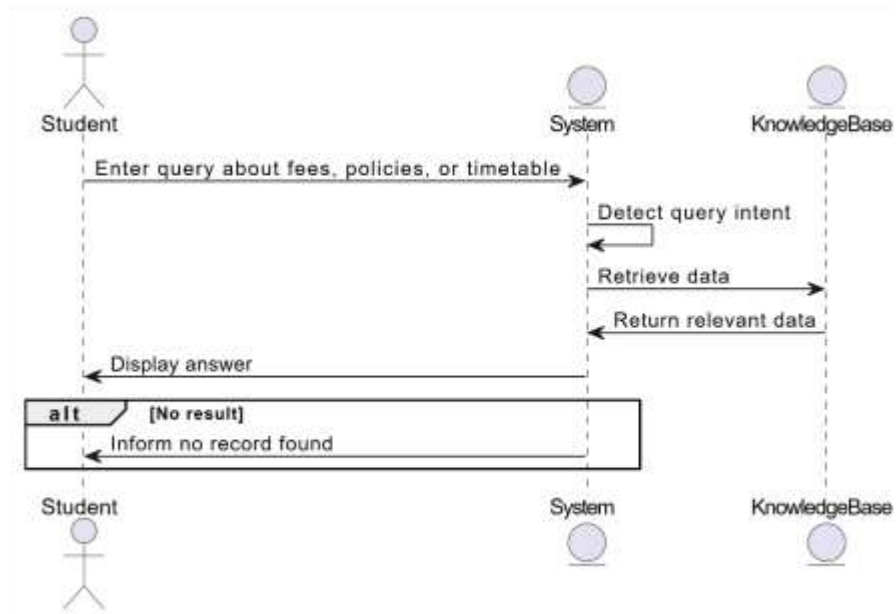


Figure 2.29: Sequence Diagram – Bilingual Query Input

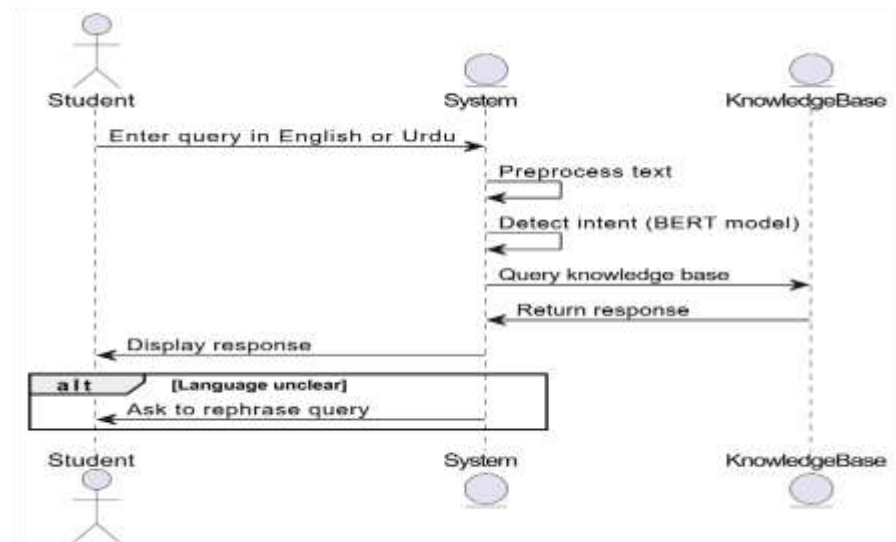


Figure 2.30: Sequence Diagram – Manage Course Information

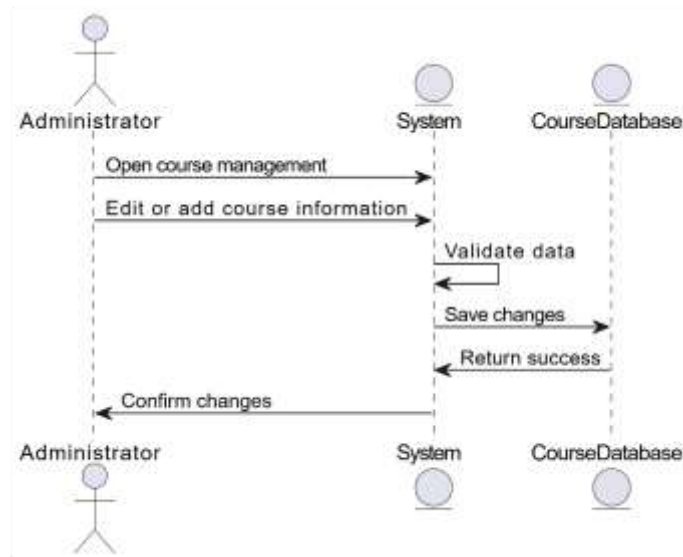


Figure 2.31: Sequence Diagram – Manage Fee Structure

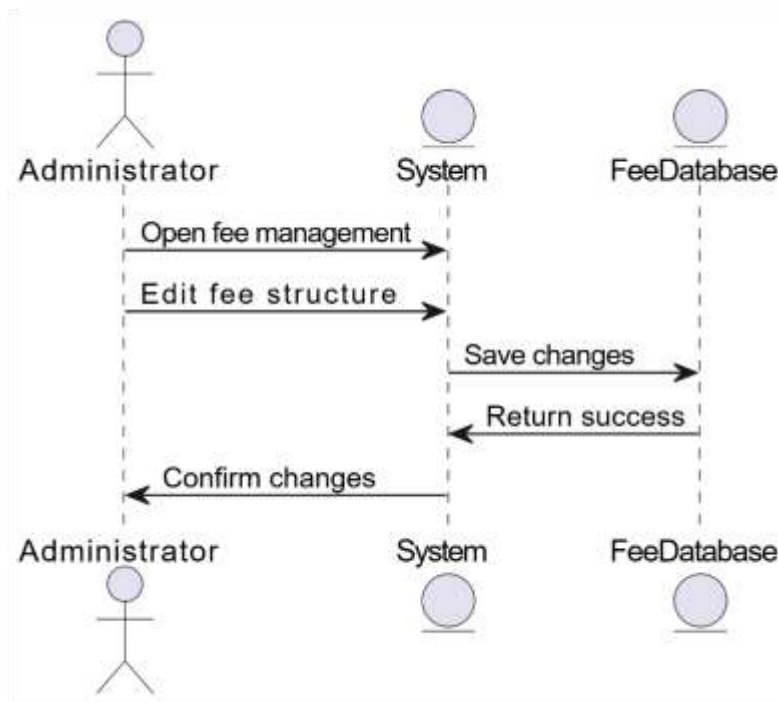


Figure 2.32: Sequence Diagram – Manage Timetables

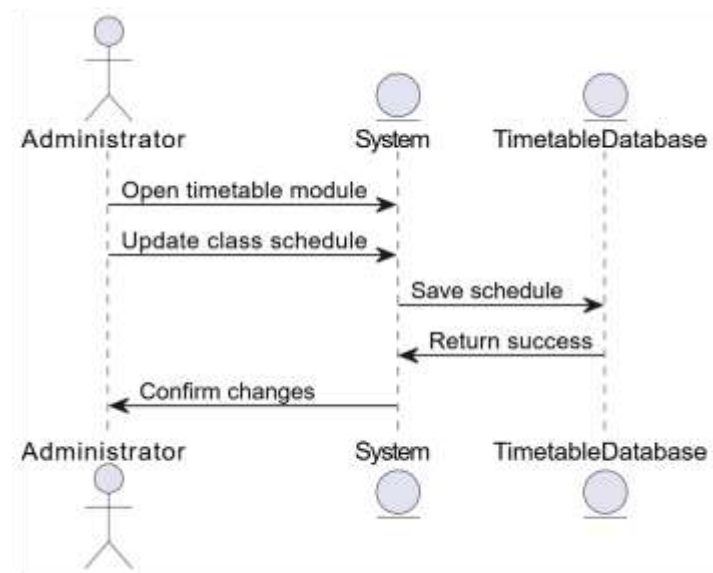


Figure 2.33: Sequence Diagram – Manage Policies & Guidelines

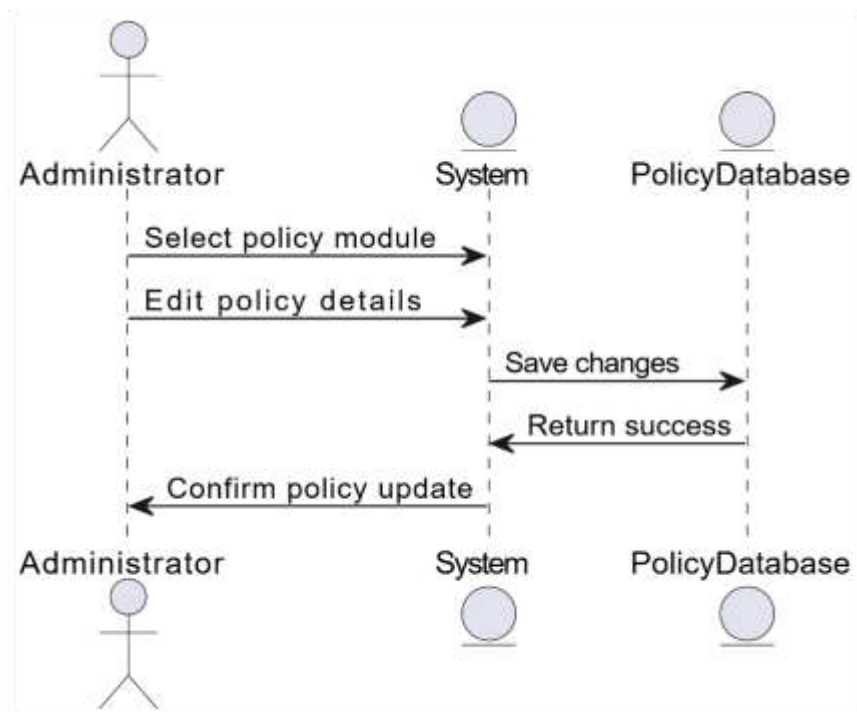


Figure 2.34: Sequence Diagram – View Student Query Logs

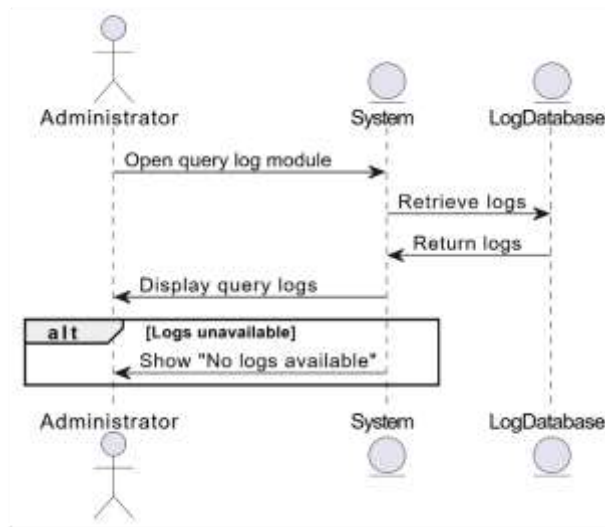


Figure 2.35: Sequence Diagram – Manage User Accounts

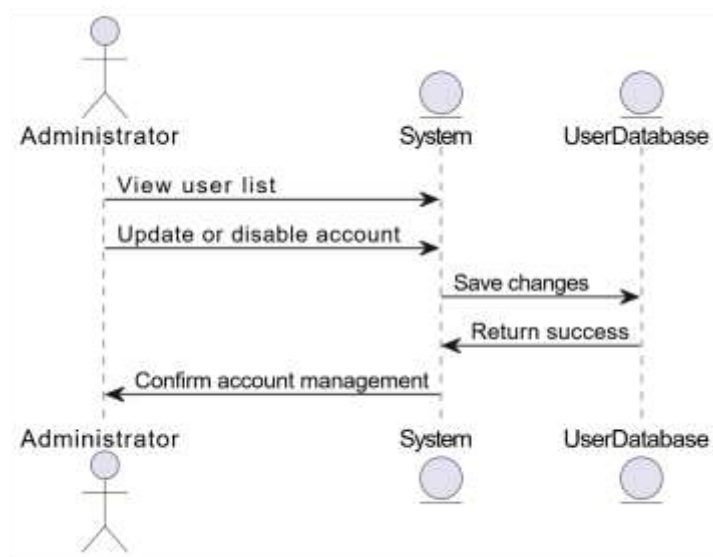


Figure 2.36: Sequence Diagram – Generate Reports

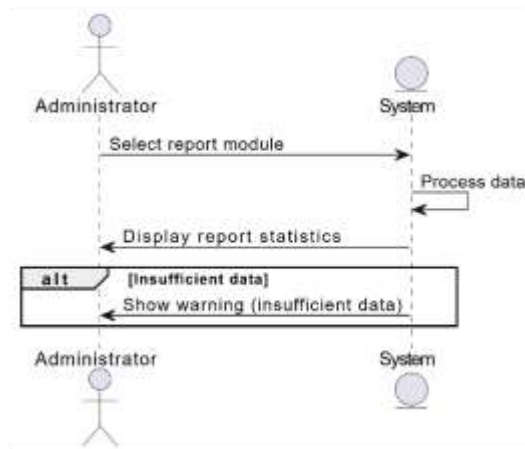
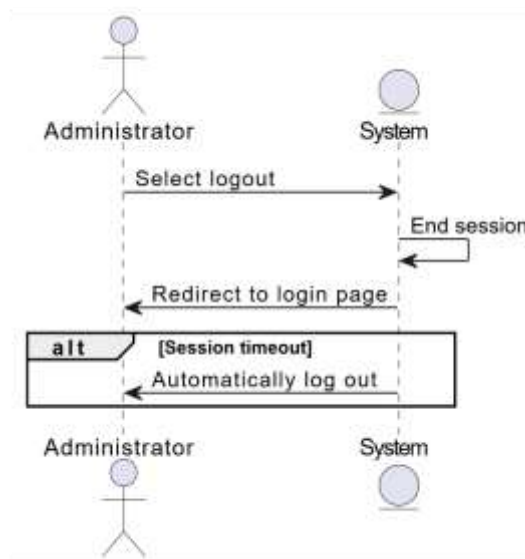


Figure 2.37 : Sequence Diagram – Admin Logout



CHAPTER – 03

SOFTWARE DESIGN DESCRIPTION

3.1 Introduction

This chapter presents the design of the proposed system and explains how the system is structured to satisfy the requirements identified in the previous chapter. It describes the overall architecture, major software components, their interactions, and the design decisions used during development of the our project.

The main purpose of the software design chapter is to show that how the component of our system are working together so that maintaining the system will become easier and in future our system is easy to upgrade or extend.

Design Overview

- **Academic Query Input:** Students can ask questions about courses, prerequisites, credit hours, fees, and policies.
- **Courses Information Search:** Students can search courses details by name.
- **Bilingual Support:** Our project will understands both **English** and **Urdu** queries.
- **Knowledge Base Retrieval:** Our system will fetch the required data from database.
- **Conversational Responses:** Our system provides easily understandable answers in a chat format.
- **Student Query History:** Students can check past queries and responses.
- **Admin Panel:** Admins can update courses, policies, schedules, and other academic info.

How Components Interact

Our system will have many core components to ensure better functionality. First of all the **chat interface** will allow students to ask questions and receive responses. Then at the backend **API** will process the requests sent by the interface, while the **NLP engine** will understand the queries and identify the intent of queries, and take out the key information such as course names or fee related terms. Then the **database** which stores structured academic data, including course details, prerequisites, schedules, and policies, for quick retrieval. Then the response **generator** will transform the data into clear and easy answers. In the last , there is an **admin panel** which allows admins to manage, update data, and maintain the system.

Requirements Traceability Matrix

Requirements Traceability Matrix (RTM) for AI-Powered Chat-Based Academic Advisor

Requirement ID	Requirement Name	Interface	Test Case	Use Case
FR-01	Student Registration	Registration Form	Test new student registration with validation	UC-01
FR-02	Student Login	Login Form	Test authentication and login flow	UC-02
FR-03	Ask Academic Query	Chat Interface	Test academic question processing	UC-03
FR-04	View Chatbot Response	Chat Window	Test display of returned answer	UC-04
FR-05	View Query History	History Page	Test retrieval of previous queries	UC-05
FR-06	Admin Login	Admin Login Panel	Test admin authentication	UC-06
FR-07	Admin Update Academic Data	Admin Dashboard	Test adding/editing course and policy data	UC-07
FR-08	Search Course by Name	Search Interface	Test course lookup by title	UC-08
FR-09	Ask Policy / Fee / Timetable Information	Chat Interface	Test policy and fee related response generation	UC-09
FR-10	Bilingual Query Input	Multilingual Chat Interface	Test English and Urdu query handling	UC-10
FR-11	Manage Course Information	Admin Dashboard	Test course CRUD operations	UC-11
FR-12	Manage Fee Structure	Admin Dashboard	Test fee record updates	UC-12
FR-13	Manage Timetables	Admin Dashboard	Test timetable management	UC-13
FR-14	Manage Policies & Guidelines	Admin Dashboard	Test policy update functionality	UC-14
FR-15	View Student Query Logs	Admin Dashboard	Test query log retrieval	UC-15
FR-16	Manage User Accounts	Admin Dashboard	Test account management functions	UC-16
FR-17	Generate Reports	Reporting Module	Test report generation	UC-17
FR-18	Admin Logout	Admin Dashboard	Test secure logout	UC-18

FR-19	Intent Detection	NLP Module	Test intent classification accuracy	UC-03, UC-09, UC-10
FR-20	Knowledge Base Retrieval	Backend + Database	Test retrieval of correct academic information	UC-03, UC-08, UC-09

Table 3.1: Requirements Traceability Matrix (RTM)

3.2 System Architecture Design

This part will provide an overview of how the parts of the software are structured. It explains how the front end, backend, NLP layer, and data storage are working together, and how data flows between them.

The architecture of our system is designed in such a way it is easy to maintain and easy to update or upgrade in future. It ensures that our system can process natural language queries, provide academic information, and provide real time responses and making sure that different parts of the system communicate in such a way they are effective and that each team member has a clear role.

3.2.1 Chosen System Architecture

The proposed system is designed using a 4-layer architecture:

1. **Presentation Layer**
2. **Application Layer**
3. **NLP Processing Layer**
4. **Data Layer**

1. Presentation Layer

Overview

The Presentation Layer is the user view of the system .It allows students and admins to interact with the ai advisor through a web-based interface.

Functionality

- Students can register and log in.
- Students can ask academic questions in English or Urdu.
- Students can view chatbot responses and query history.
- Admins can log in to the admin panel and update academic records.

Technologies Used

- HTML, CSS, JavaScript or React

2. Application Layer

Overview

The system's business logic is contained in the application layer. It handles front-end requests and arranges communication with the database and NLP module.

Functionality

- Responds to requests for login and registration
- Responds to students' academic inquiries
- Sends the NLP engine queries.
- Retrieves scholarly information from the database
- Produces well-organized answers
- Preserves system logs and query history

Technologies Used

- Python
- Flask / FastAPI / Django
- REST APIs with JSON responses

3. NLP Processing Layer

Overview

This layer is responsible for understanding the query entered by the user. It translates the sentence, identifies the query, and extracts the relevant entities.

Functionality

- Query preprocessing
- Tokenization and text normalization
- Intent detection
- Multilingual query handling in English and Urdu

Technologies Used

- Multilingual BERT
- Transformers library
- Python NLP utilities

4. Data Layer

Overview

This layer stores all structured academic information and user records. This is the source of knowledge for the project.

Functionality

- Stores course information
- Stores prerequisite relationships
- Stores fee structures
- Stores class schedules and timetables
- Stores academic policies and thesis guidelines
- Stores student query history and admin logs

Technologies Used

- MySQL / PostgreSQL / MongoDB
- Structured knowledge base tables
- Secure storage for user and admin records

3.3 Architectural Diagram

The architectural design of our system show the relationship between the student interface, admin panel, backend services, NLP engine, and academic database.

Figure 3.1: Architecture Diagram of Academic Advisor

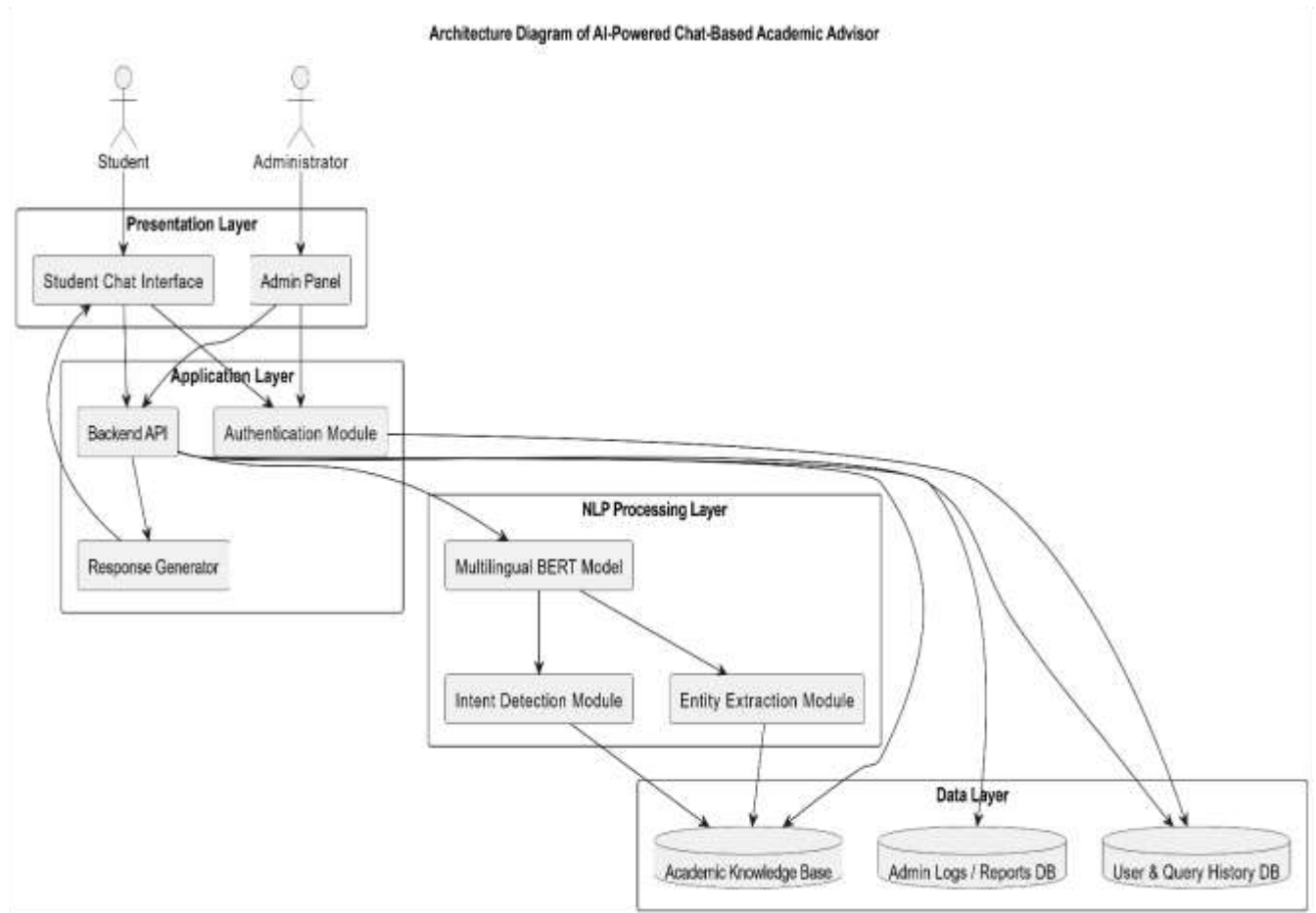


Figure 3.1: Architecture Diagram of AI-Powered Chat-Based Academic Advisor

3.4 Class Diagram

Class model I of our system includes entities such as **Student**, **Admin**, **Query**, **Response**, **Course**, **Policy**, **Timetable**, **Knowledgebase**, and **Chatbot Engine**.

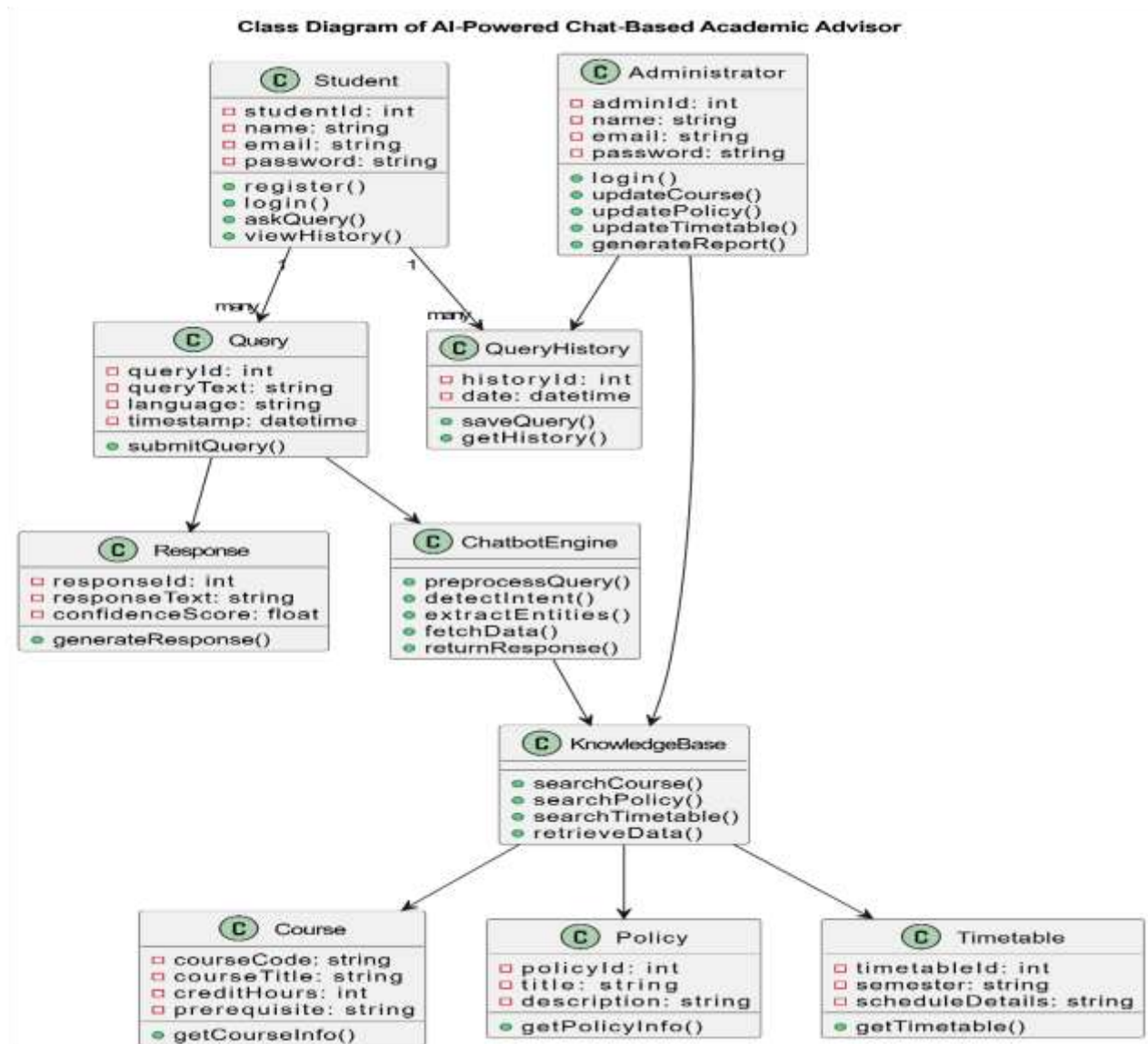


Figure 3.2: Class Diagram of AI-Powered Chat-Based Academic Advisor

3.5 Component Description

Component	Description	Technologies
Student Chat Interface	Web-based chat interface for students to ask academic questions	HTML, CSS, JavaScript / React
Admin Panel	Dashboard for admin login, data management, and report generation	React / HTML, CSS, JavaScript
Backend API	Handles requests, business logic, and communication between modules	Python, Flask / FastAPI / Django
Authentication Module	Manages student and admin login, registration, and session control	Firebase Auth / JWT / Session-based auth
Multilingual BERT Model	Processes student queries in English and Urdu	Transformers, Python
Intent Detection Module	Identifies whether the query is about course, fee, policy, timetable, etc.	BERT fine-tuning, Python
Entity Extraction Module	Extracts course names and important query terms	NLP pipeline
Academic Knowledge Base	Stores course details, prerequisites, fee structure, policies, and timetables	MySQL / PostgreSQL / MongoDB
Query History Database	Stores previous student queries and chatbot responses	SQL / NoSQL DB
Admin Logs / Reports Module	Stores usage logs and system reports	Database + reporting utilities
Response Generator	Converts retrieved data into readable chatbot responses	Python backend logic

Table 3.2: Component Description

3.6 Activity Diagrams

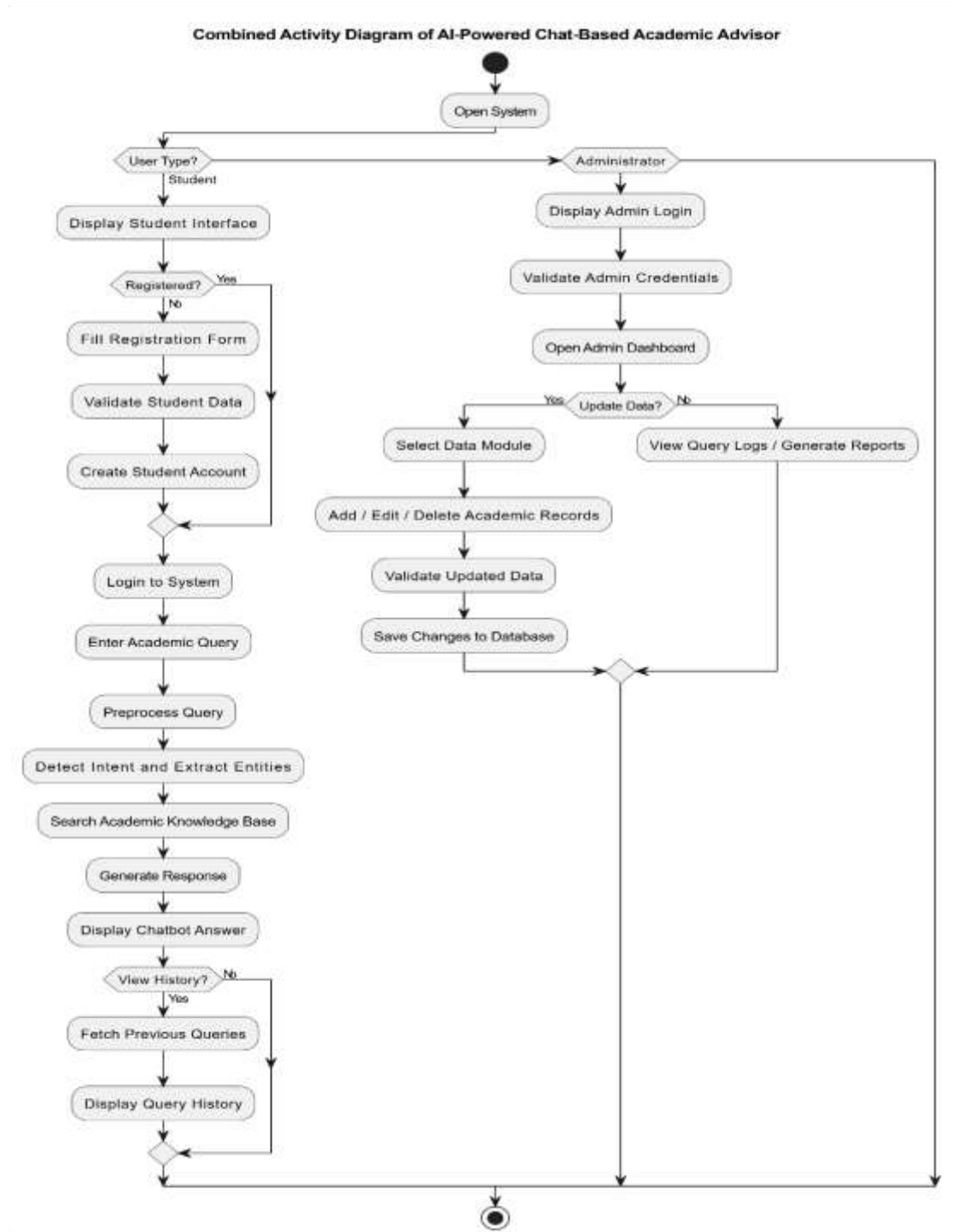


Figure 3.3: Combined Activity Diagram of AI-Powered Chat-Based Academic Advisor

3.7 Interface/Screen Design

3.7.1 Login Screen



Login

Welcome to Quaid-i-Azam University online system.

Email:

Password:

[Forgot Password?](#)

3.7.2 Ask Academic Query

English | اردو

Suggested Queries

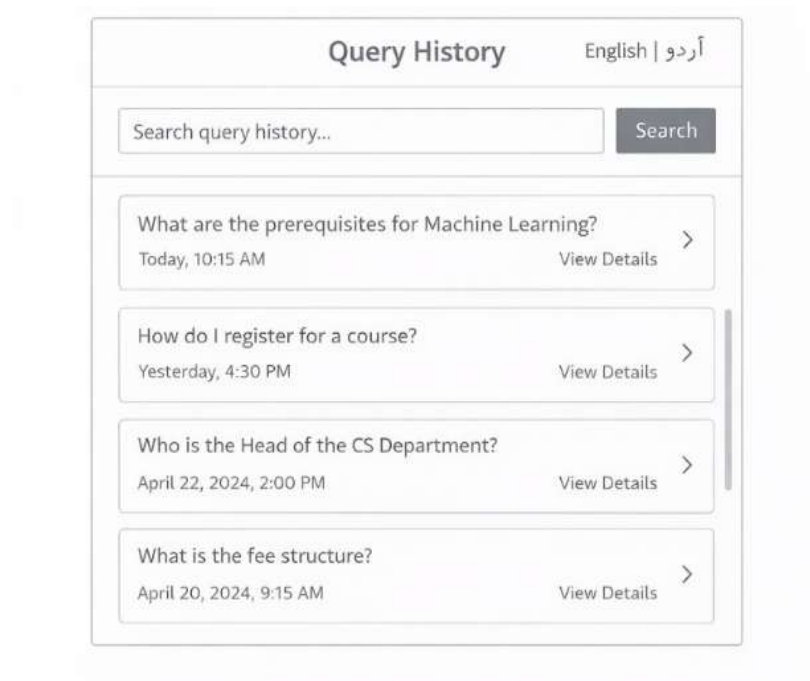
What is the fee structure?	How do I register for a course?
Who is the HOD of the CS Department?	What are the prerequisites for Machine Learning?

What are the prerequisites for Machine Learning?

The prerequisites for Machine Learning include "Data Structures" and "Probability and Statistics".

Type your query here...

3.7.3 View Query History



3.7.4 Admin Login and Dashboard



3.7.5 Admin Manage Data

QUAID-I-AZAM UNIVERSITY ISLAMABAD
Established: 1973

Admin User

Logout

Dashboard

Activity Logs

Manage Data

Admin Data Management

COURSE INFORMATION

Course Title

Course Description

Prerequisites

Upload Supporting Document

Save Changes

FEE STRUCTURE

Select Semester

Select Program

Fee Amount

Upload Fee Structure

Update Fee Structure

TIMETABLE

Select Semester

Select Program

Upload Timetable Document

Add Class Schedule

Save Timetable

Navigation: PreviousNext

Previous

Next

3.7.6 Admin Monitor Activity



QUAID-I-AZAM UNIVERSITY ISLAMABAD

English/Urdu

AdminLogout

Dashboard

Manage Data

Activity Logs

AI Model Update

Settings

USER ACTIVITY LOGS

FILTER BY DATE

Start Date → End Date

FILTER BY USER

All Users

Filter

Export Logs

USER NAME	EMAIL	QUERY COUNT	LAST LOGIN	LAST QUERY	DURATION
Ali Ahmad	ali@example.com	12	24-04-2024 10:15 AM	"Prerequisites for ML?"	15 min
Sara Khan	sara@example.com	5	23-04-2024 4:30 PM	"Fee Structure"	8 min
Usman Tariq	usman@example.com	28	23-04-2024 11:00 AM	"Admission Criteria"	45 min
Zainab Bibi	zainab@example.com	2	22-04-2024 9:20 AM	"Hostel Availability"	3 min
Hamza Ali	hamza@example.com	15	22-04-2024 2:15 PM	"Exam Schedule"	22 min

Showing 1-5 of 72 entries

Previous123...15Next

3.8 Summary

This chapter explained the software design of the proposed academic advisor system. Basically it describe the architecture ,layers ,class diagram ,components and activity flow.

CHAPTER: 04

SOFTWARE TEST DOCUMENTATION

4.1 Introduction

In this chapter, one can learn about the testing plan of **the AI-Powered Chat-Based Academic Advisor of the Computer Science Department at Quaid-i-Azam University (QAU)**. Testing is intended to ensure that the system is functioning as per the needs that were identified in the previous chapters and has the ability to offer quality academic assistance to the students. Because the project is founded on a chatbot, which will answer academic questions, it is not only essential to test the basic functions of a software (e.g., the ability to log in and register), but also to test the query processing, information search, bilingual input processing, and administration functions.

The process of testing is supposed to verify that the system gives appropriate answers, the invalid inputs are handled and the system is able to continue to be used by students, as well as, administrators.

4.1.1 System Overview

The **AI-Powered Chat-Based Academic Advisor** is an academic support web-based platform created in the Computer Science Department of Quaid-i-Azam University. The students can communicate with a chatbot and see the information concerning the courses, prerequisites, fee structure, class schedules, academic regulations, and thesis structures.

The system employs a **Multilingual BERT-based NLP model** to comprehend student queries both in English and Urdu. It then sources the sought information in an organized academic knowledge base and yields a response in form of conversations. The system also has a student facing feature as well as an administrative panel where academic data is updated, query logs are viewed and reports are generated..

4.1.2 Test Approach

Testing basically focuses on verifying functional behavior of all the major use cases of the system. Both **functional testing** and selected **non-functional testing** are considered. The main goal is to ensure that our sytem responds correctly, handles invalid input safely, and performs reliably under normal usage conditions.

The testing process is primarily based on **manual execution of test cases**,

4.1.3 Features to be Tested

1. **Student Registration (UC1)**
2. **Student Login (UC2)**
3. **Ask Academic Query (UC3)**
4. **View Chatbot Response (UC4)**
5. **View Query History (UC5)**
6. **Admin Login (UC6)**
7. **Admin Update Academic Data (UC7)**
8. **Search Course by Name (UC8)**
9. **Ask Policy / Fee / Timetable Information (UC9)**
10. **Bilingual Query Input (UC10)**
11. **Manage Course Information (UC11)**
12. **Manage Fee Structure (UC12)**
13. **Manage Timetables (UC13)**
14. **Manage Policies & Guidelines (UC14)**
15. **View Student Query Logs (UC15)**
16. **Manage User Accounts (UC16)**
17. **Generate Reports (UC17)**
18. **Admin Logout (UC18)**

4.1.4 Features Not to be Tested

1. **Heavy Load and Stress Testing**
2. **Advanced Security Penetration Testing**
3. **Cross-Browser Legacy Support**
4. **Long-Term Monitoring**

4.2 Testing Tools and Environment

Environment

- Local host
- Web interface
- Nodejs/expressjs backend server
- Academic database
- NLP integration

Tools

- Postman
- VS Code
- Test validation sheets for use case.

4.3 Test Cases

4.3.1 Test Case: Student Registration (UC1)

TEST ID	T-01	
USE CASE	UC1 – Student Registration	
PURPOSE	Verify that a new student can create an account successfully	
PRE-CONDITIONS	Student is not already registered. Registration page is accessible.	
POST-CONDITIONS	Student account is created and stored in the system.	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Register with correct details	Name: “Ali Khan”, Email: “ ali@example.com ”, Password: “Password123”	Account created successfully and redirect to login
Invalid: Existing email	Email already stored in database	Error message: “Email already registered”
Invalid: Password too short	Password: “123”	Error message: “Password must be at least 6 characters”
Invalid: Password mismatch	Password and confirm password do not match	Error message shown
Invalid: Empty fields	All fields empty	Error message: “All fields are required”

S4.3.2 Test Case: Student Login (UC2)

TEST ID	T-02	
USE CASE	UC2 – Student Login	
PURPOSE	Verify that registered students can log in successfully	
PRE-CONDITIONS	Student account exists	
POST-CONDITIONS	Student is authenticated and redirected to dashboard	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Correct login	Registered email and password	Login successful
Invalid: Wrong password	Correct email, wrong password	Error message: “Invalid email or password”
Invalid: Unregistered email	Unknown email	Error message: “Account does not exist”
Invalid: Empty fields	Empty input	Error message asking to enter credentials

4.3.3 Test Case: Ask Academic Query (UC3)

TEST ID	T-03
USE CASE	UC3 – Ask Academic Query
PURPOSE	Verify that student queries are processed correctly
PRE-CONDITIONS	Student is logged in

POST-CONDITIONS	System retrieves and displays academic answer	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Course prerequisite query	What are the prerequisites for data structure?	Correct information displayed
Valid: Credit hour query	How many credit hours a student can take in a semester	Relevant data to be displayed
Invalid: Unclear query	Tell me that thing	System asks students to rephrase
Invalid: Unknown topic	Query unrelated to available data	Output of system No relevant information found

4.3.4 Test Case: View Chatbot Response (UC4)

TEST ID	T-04	
USE CASE	UC4 – View Chatbot Response	
PURPOSE	To check our system responses are shown correctly	
PRE-CONDITIONS	Query has processed	
POST-CONDITIONS	Academic data or answer to be displayed	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Successful answer display	Returned chatbot text	Response to be seen on system chat window
Valid: Multi-line response	Long explanation	Maintain proper formatting
Invalid: Empty response	missing backend response	Error message shown

4.3.5 Test Case: View Query History (UC5)

TEST ID	T-05	
USE CASE	UC5 – View Query History	
PURPOSE	Verify that students can view previous queries	
PRE-CONDITIONS	Student is logged in and history exists	
POST-CONDITIONS	Query history is displayed	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Existing history	Student with saved records	Previous queries and responses displayed
Invalid: No history	New user	Message: “No history found”
Invalid: Database error	History retrieval fails	Error message shown

4.3.6 Test Case: Admin Login (UC6)

TEST ID	T-06
USE CASE	UC6 – Admin Login
PURPOSE	Verify that administrator can log in securely
PRE-CONDITIONS	Admin account exists
POST-CONDITIONS	Admin enters dashboard

TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Correct admin credentials	Valid admin email and password	Dashboard access granted
Invalid: Wrong credentials	Incorrect password	Access denied
Invalid: Empty fields	No credentials entered	Validation error message

4.3.7 Test Case: Admin Update Academic Data (UC7)

TEST ID	T-07	
USE CASE	UC7 – Admin Update Academic Data	
PURPOSE	Check admin can update academic records	
PRE-CONDITIONS	Admin logged in	
POST-CONDITIONS	Academic data updated successfully	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Update course info	New course title added	Course title successfully
Valid: Update fee info	fee structure updated	Fee structure updated
Invalid: Missing required field	Incomplete academic data entry	error shown

4.3.8 Test Case: Search Course by Name (UC8)

TEST ID	T-08	
USE CASE	UC8 – Search Course by Name	
PURPOSE	Verify that course search works correctly	
PRE-CONDITIONS	Student is logged in	
POST-CONDITIONS	Course details displayed	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Search by full course name	“Data Structures”	Correct course displayed
Valid: Search by course code	“CS-201”	Matching course data shown
Invalid: Unknown course	“ABC Course”	Message: “Course not found”
Invalid: Empty search	No input	Prompt to enter course name

4.3.9 Test Case: Ask Policy / Fee / Timetable Information (UC9)

TEST ID	T-09	
USE CASE	UC9 – Ask Policy / Fee / Timetable Information	
PURPOSE	Verify policy and schedule related question handling	
PRE-CONDITIONS	Student is logged in	
POST-CONDITIONS	Requested information displayed	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Fee query	“What is the semester fee?”	Correct fee information shown
Valid: Timetable query	“Show class timetable for 6th semester”	Matching timetable displayed
Valid: Policy query	“What is the attendance policy?”	Relevant policy returned

Invalid: Unknown query	Policy not stored in system	“No record found” message
------------------------	-----------------------------	---------------------------

4.3.10 Test Case: Bilingual Query Input (UC10)

TEST ID	T-10	
USE CASE	UC10 – Bilingual Query Input	
PURPOSE	Check that system can process English and Urdu queries	
PRE-CONDITIONS	Student is logged in	
POST-CONDITIONS	Queries translated or response shown	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: English query	What are the prerequisites for AI?	answer shown correctly
Valid: Urdu query	AI ke prerequisites kya Hain?	answer shown correctly
Invalid: Mixed unclear input	Unclear multilingual sentence	System asks to rephrase

4.3.11 Test Case: Manage Course Information (UC11)

TEST ID	T-11	
USE CASE	UC11 – Manage Course Information	
PURPOSE	Verify admin can add, edit, and delete course data	
PRE-CONDITIONS	Admin logged in	
POST-CONDITIONS	Course records updated	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Add course	New course entry	Course saved
Valid: Edit course	Updated prerequisite	Changes saved
Invalid: Missing fields	No course code/title	Validation error

4.3.12 Test Case: Manage Fee Structure (UC12)

TEST ID	T-12	
USE CASE	UC12 – Manage Fee Structure	
PURPOSE	Check whether admin can update fee information	
PRE-CONDITIONS	Admin logged in	
POST-CONDITIONS	Fee records updated	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Edit fee structure	Updated semester fee	Saved successfully
Invalid: Incorrect amount format	Text instead of numeric value	Validation error

4.3.13 Test Case: Manage Timetables (UC13)

TEST ID	T-13
USE CASE	UC13 – Manage Timetables

PURPOSE	Verify admin can maintain timetable records	
PRE-CONDITIONS	Admin logged in	
POST-CONDITIONS	Timetable updated	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Add timetable entry	Semester + course + timing	Timetable saved
Invalid: Invalid schedule format	Missing day/time info	Validation error

4.3.14 Test Case: Manage Policies & Guidelines (UC14)

TEST ID	T-14	
USE CASE	UC14 – Manage Policies & Guidelines	
PURPOSE	Verify admin can update policy data	
PRE-CONDITIONS	Admin logged in	
POST-CONDITIONS	Policy information updated	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Add new policy	Attendance policy text	Policy saved
Invalid: Empty content	Blank policy field	Validation message

4.3.15 Test Case: View Student Query Logs (UC15)

TEST ID	T-15	
USE CASE	UC15 – View Student Query Logs	
PURPOSE	Verify admin can inspect query logs	
PRE-CONDITIONS	Admin logged in	
POST-CONDITIONS	Query logs displayed	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: View logs	Existing stored logs	Log table displayed
Invalid: No logs	Empty log table	Message indicating no records

4.3.16 Test Case: Manage User Accounts (UC16)

TEST ID	T-16	
USE CASE	UC16 – Manage User Accounts	
PURPOSE	Check whether admin can manage student accounts	
PRE-CONDITIONS	Admin logged in	
POST-CONDITIONS	account changes updated	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Disable account	Select a account	updated
Valid: Edit account info	Update student record	Updated and saved
Invalid: Unauthorized action	Invalid admin access	Access denied

4.3.17 Test Case: Generate Reports (UC17)

TEST ID	T-17
----------------	-------------

USE CASE	UC17 – Generate Reports	
PURPOSE	Verify admin can generate system reports	
PRE-CONDITIONS	Admin logged in	
POST-CONDITIONS	Report generated and shown	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Generate usage report	Date range selected	Report displayed
Invalid: No data in range	Empty result set	Informative warning shown

4.3.18 Test Case: Admin Logout (UC18)

TEST ID	T-18	
USE CASE	UC18 – Admin Logout	
PURPOSE	Check logout functionality	
PRE-CONDITIONS	Admin logged in	
POST-CONDITIONS	Session terminated	
TEST SCENARIO	TEST DATA	EXPECTED OUTPUT
Valid: Click logout	Active admin session	Redirect to login page
Valid: Session timeout	Inactive session	Automatic logout

4.4 Summary

It covered the system overview, testing approach, environment, tools, and detailed test cases for the major use cases for our project.

The purpose of these test cases are to verify that functionality of our system is correct for both student and admin login , handles both valid and invalid input properly and give proper output.

CHAPTER: 05

CONCLUSION

5.1 Conclusion

This document has the implementation of the design and development of My FYP i.e. **AI-Powered Chat-Based Academic Advisor in the Computer Science Department at Quaid-i-Azam University (QAU)**. The main objective for this project is to give the student an accessible and user friendly experience without depending on the manual system or communication.

The source of information involved a structured academic body of knowledge, whereas the **NLP** method to understand and process natural language queries was a **Multilingual BERT-based** approach. All through the project, emphasis was laid on the system design, requirement analysis, software architecture, use case modeling, sequence diagrams, class design, and testing. The system was more comprehensive as the student-side and the admin-side modules were included in it, as it does not answer the questions only, but also lets the academic records be managed and updated by the means of an administration panel.

The testing procedures was to check the key scenarios of the system such as student registration ,login ,admin login search courses academic queries e.t.c. These tests were conducted to ensure that the proposed system can provide accurate answers and be effectively tested by its users. It provides an effective way of enhancing communication between the academic department and it can as well be expanded in the future to accommodate more departments or additional services at the university level.

References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.
- [2] T. Wolf et al., “Transformers: State-of-the-Art Natural Language Processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.
- [3] J. Brownlee, *Natural Language Processing with Python: Quick Start Guide for Beginners*. Machine Learning Mastery, 2017.
- [4] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2016.
- [5] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 8th ed. New York, NY, USA: McGraw-Hill, 2015.
- [6] D. Jurafsky and J. H. Martin, *Speech and Language Processing*. Draft online edition, Stanford University, 2023.